VERILOG · HOMEWORK

FSM 기반 Control 설계

D-FF · Setup / Hold · CDC · Glitch · Moore vs Mealy
Homework-1 (S0 ~ S4) · Homework-2 (FSM_RUN_STOP)

정안성 · 2026. 10. 08

목차

00

개념 정리

01 **D-FF 구조** — SR Latch → Gated D Latch → Master-Slave — p.3 – 5

02 **Setup · Hold** — 정의 · 발생 원인 · Slack · Synchronizer — p.6 – 9

03 **CDC** — 정의 · 문제 유형 · Single-bit · Multi-bit — p.10 – 14

04 **Glitch** — 발생 원인 · FF 필요 여부 — p.15 – 16

05 **Moore vs Mealy** — 출력 결정 방식 · 응답 시점 비교 — p.17 – 18

설계 · 검증

06 **Homework-1 · FSM_S3** — State Diagram · TB · 시뮬레이션 · Latch · 보드 — p.19 – 35

07 **Homework-2 설계** — State Diagram · Block Diagram · 코드 연결 · RTL — p.36 – 42

08 **Homework-2 검증** — 기존 TB · 시나리오 3종 파형 — p.43 – 46

09 **검증 방법론 평가** — 평가표 · 결함 주입 7종 — p.47 – 48

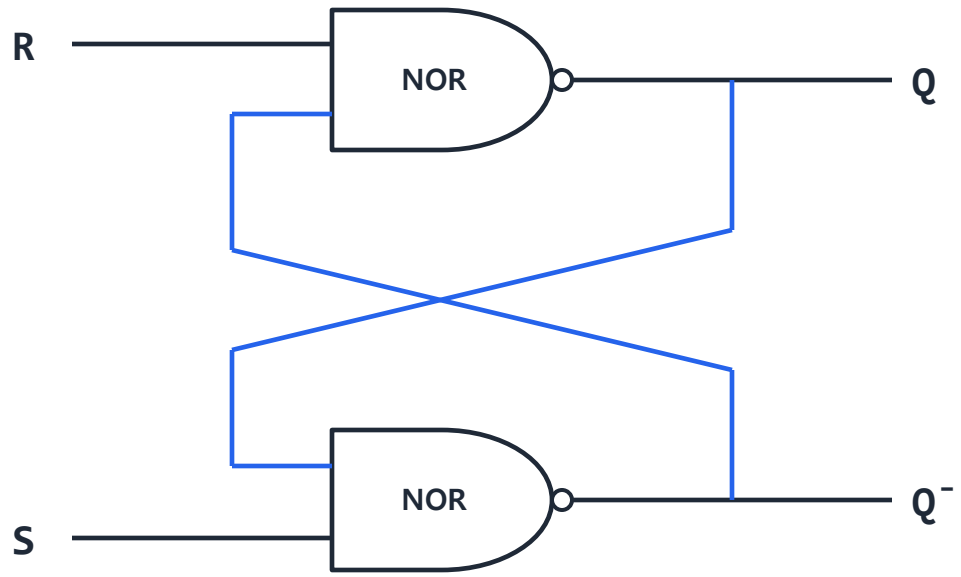
10 **새 TB 제안** — Golden Model · 채점 항목 C1 ~ C4 · 실행 결과 — p.49 – 60

11 **결론** — 결론 · 고찰 — p.61

SR Latch Oscillates When S and R Release Together

01

NOR SR Latch

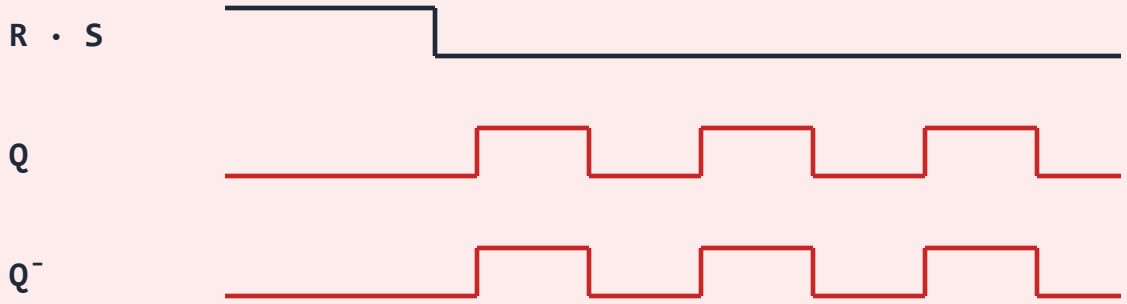


Q · Q⁻가 서로의 입력으로 되돌아감 → 값 유지

Truth Table

R	S	Q	Q⁻	동작
0	0	Q	Q⁻	HOLD
0	1	1	0	SET
1	0	0	1	RESET
1	1	0	0	금지 (Q = Q⁻)

R S = 11 → 00 동시 변경 시

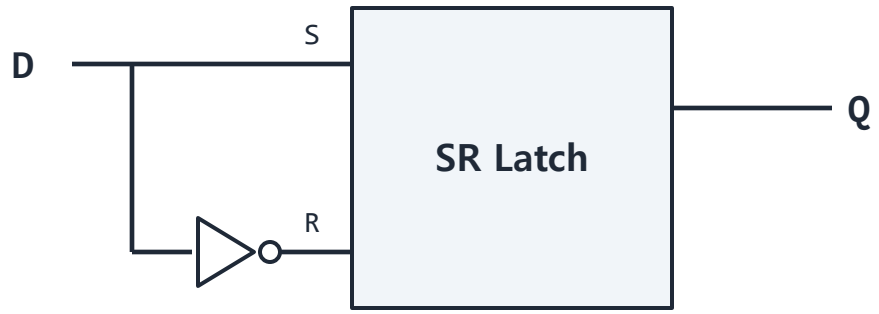


두 NOR 동시 0 → 동시 1 → 동시 0 반복 = 발진

Enable Gate Restores Memory to the D Latch

#3 D Latch

S = D, R = D⁻ → R S = 11 입력 불가

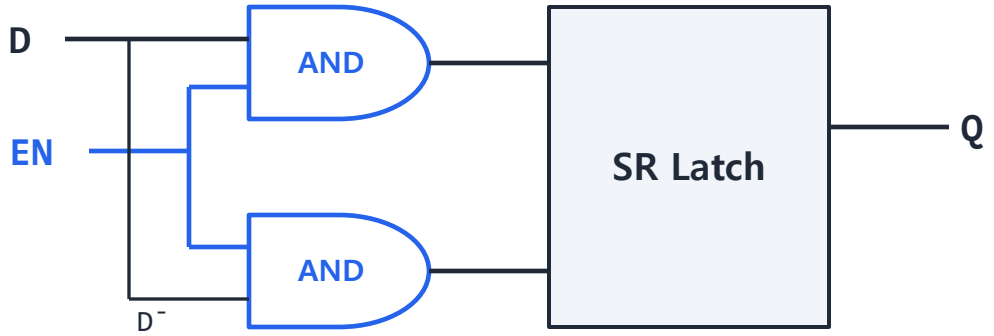


D	S R	Q
0	0 1	0
1	1 0	1

HOLD 입력 (00) 없음 → Q = D 항상 따라감
→ 메모리 기능 상실

#4 Gated D Latch

S · R 앞에 AND + EN 추가

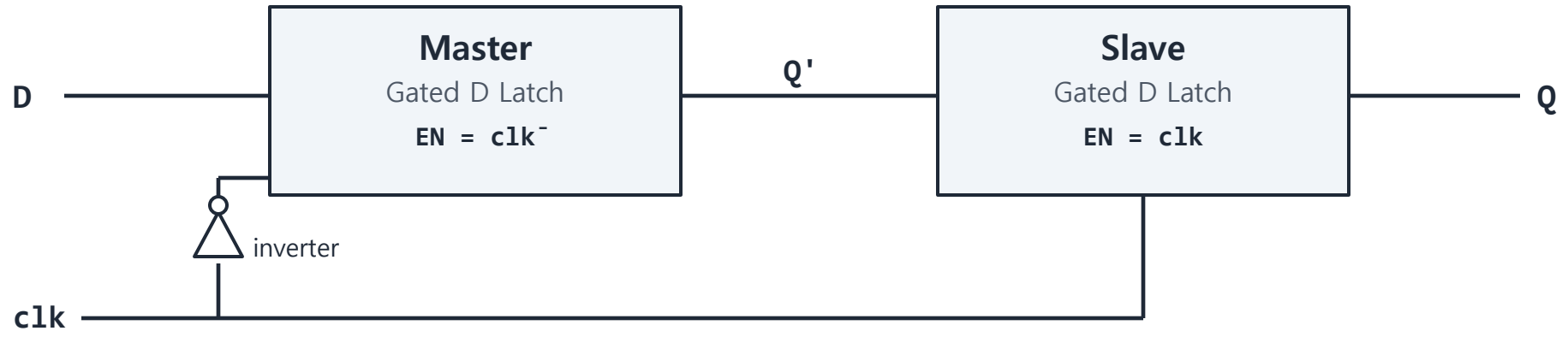


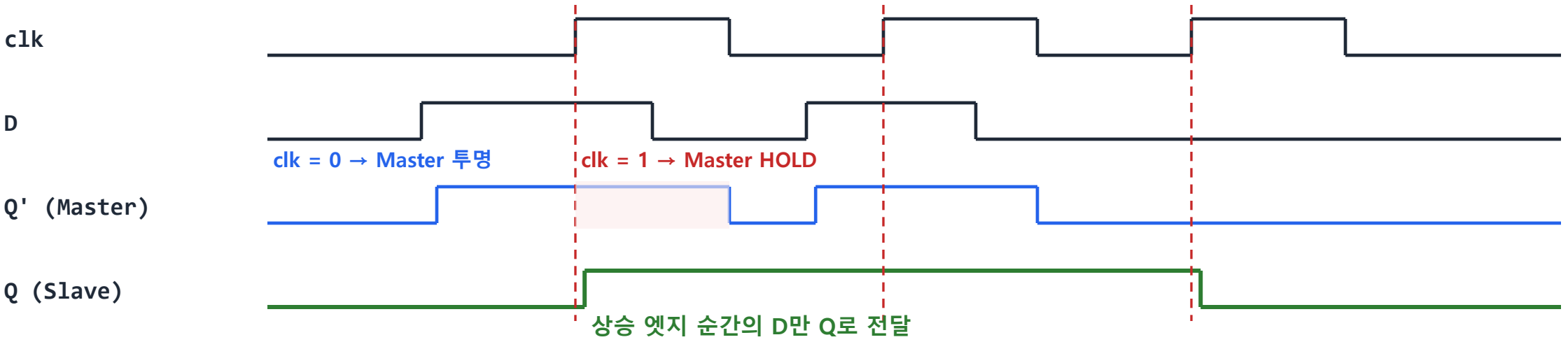
EN	D	Q
0	x	HOLD
1	0 / 1	= D

EN = 0 → S = R = 0 → HOLD 복구
EN = 1 동안 D 변화 전부 통과 (level 동작)

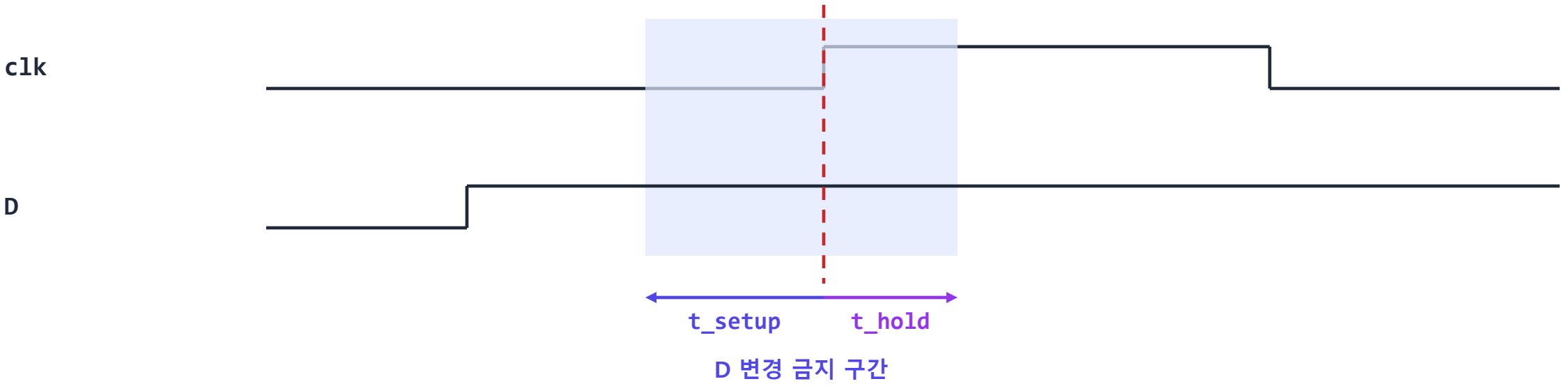
Two Gated Latches Capture D at the Rising Edge

01





D Must Stay Still Around the Clock Edge



Setup Time

상승 엣지 이전 D 유지 시간

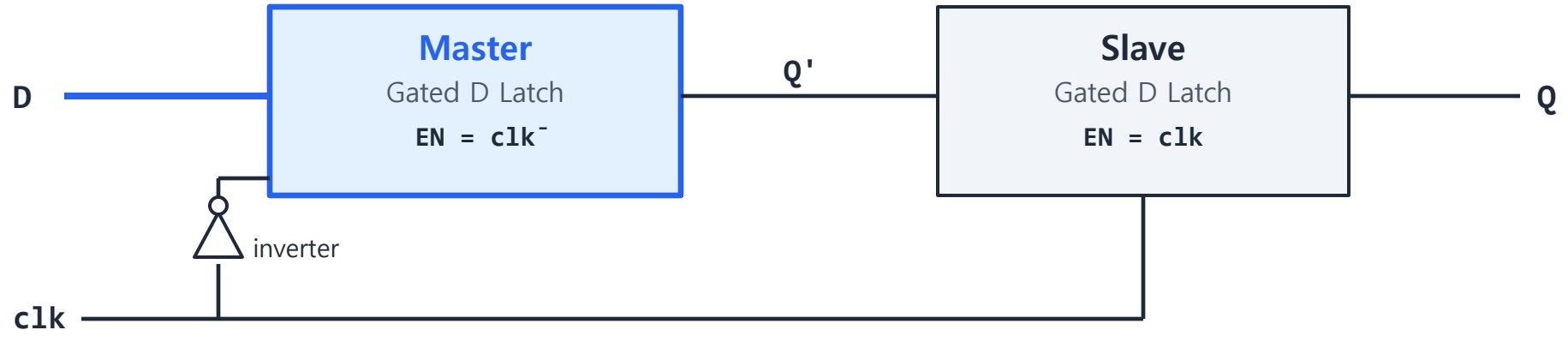
Hold Time

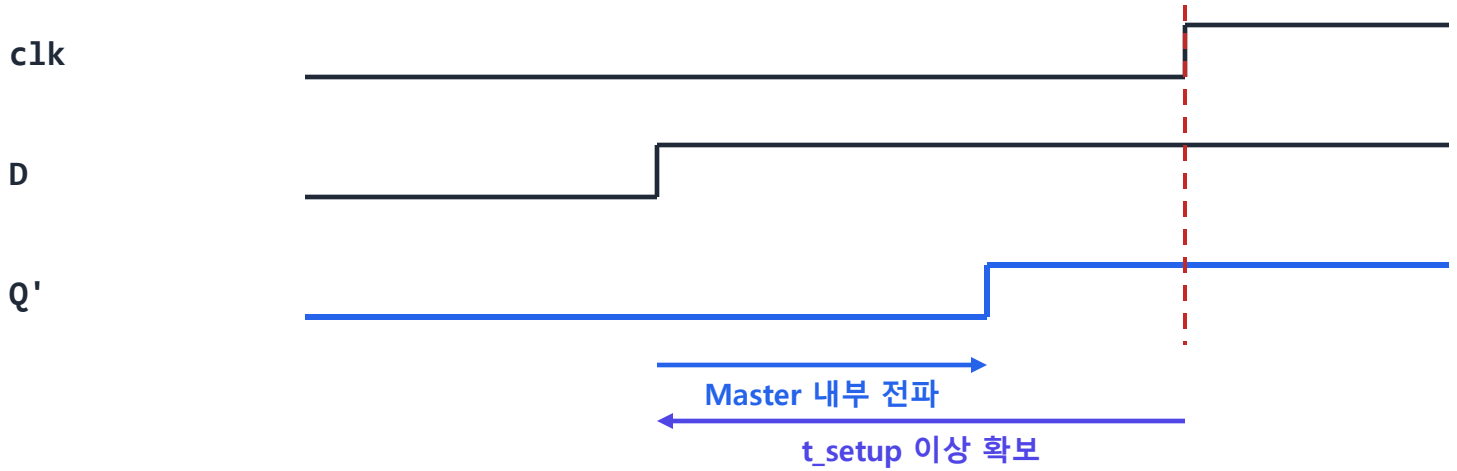
상승 엣지 이후 D 유지 시간

위반 시

Master Latch 출력 0 / 1 미결정 = Metastability

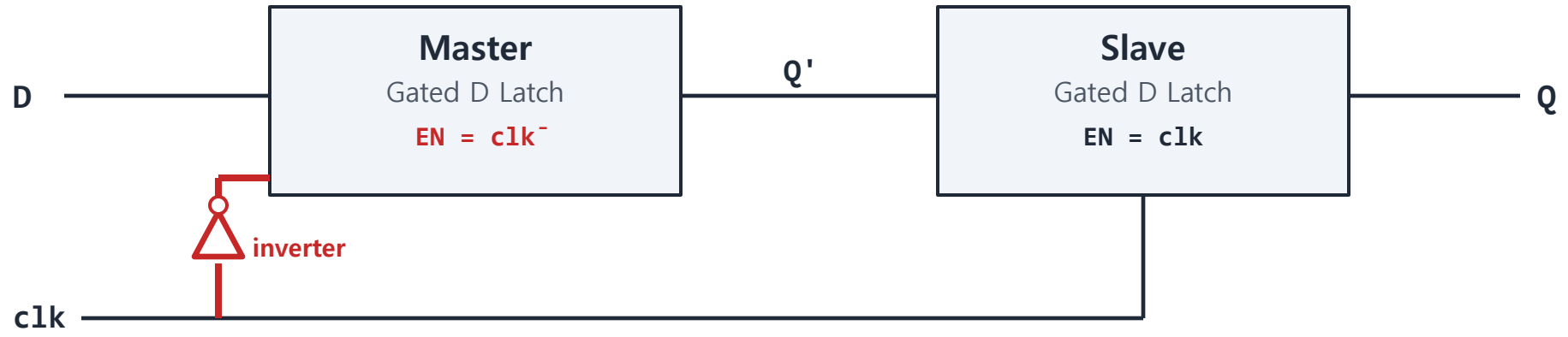
Setup Time Lets the Master Settle Before It Closes

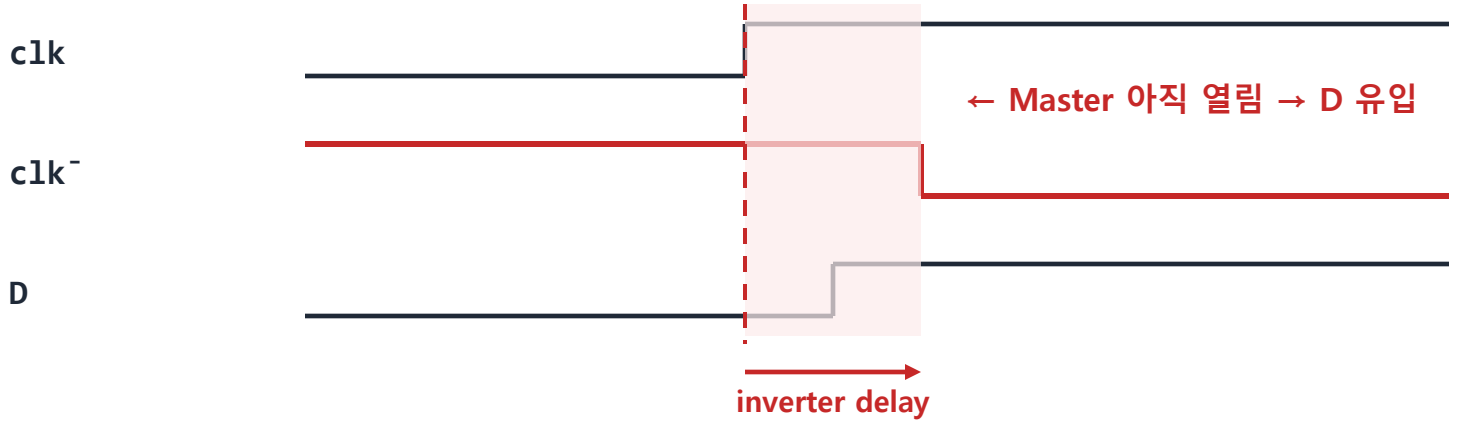




- D → Master gate → feedback loop 안정까지 delay 존재
- 상승 엣지 = Master EN 차단 시점
- 차단 전 loop 미안정 → Q' 중간 전압 고정
- Setup Time ≥ Master 내부 전파 delay

Hold Time Covers the Late Closing of the Master





- Master EN = clk⁻ → inverter 통과
- clk 상승 후 inverter delay 동안 Master 열림 유지
- 이 구간 D 변경 → 새 값이 Q'로 유입
- Hold Time ≥ clk⁻ 지연 – D 경로 delay
- ideal (delay 0) 조건일 경우 Hold = 0

Slack Sets the Clock Limit, Synchronizer Guards Async Inputs

02

Setup 조건

- T_clk ≥ T_cq + T_comb(max) + T_setup
- Slack (margin) = T_clk − (T_cq + T_comb + T_setup)
- Negative Slack → 조합 경로 분할 (pipeline) 또는 클럭 감소

Hold 조건

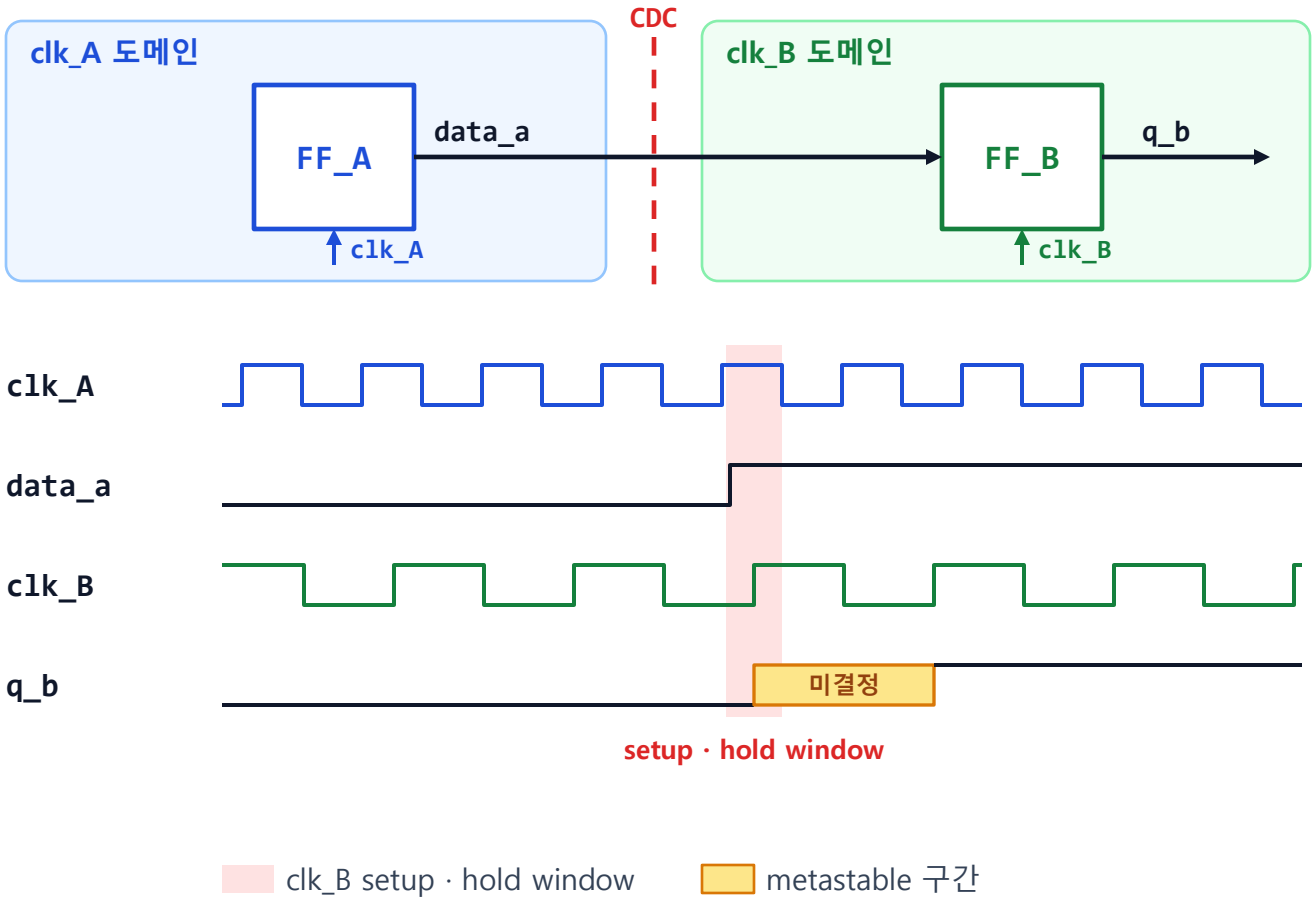
- T_cq + T_comb(min) ≥ T_hold
- 클럭 주파수와 무관
- Hold 위반 → 짧은 경로에 delay 추가

비동기 입력

- 스위치 · 외부 신호 = 클럭과 무관한 변화
- Setup · Hold 위반 피할 수 없음
- 2-FF Synchronizer로 metastability 확률 감소

CDC Path Has No Guaranteed Setup and Hold Margin

03



정의

- CDC (Clock Domain Crossing) = 서로 다른 클럭으로 동작하는 FF 사이의 신호 전달
- 송신 FF = clk_A, 수신 FF = clk_B

발생 원인

- clk_A · clk_B 위상 관계 고정 불가 (비동기 · 주파수 상이)
- data_a 변경 시점이 clk_B setup · hold window와 겹칠 확률 항상 존재

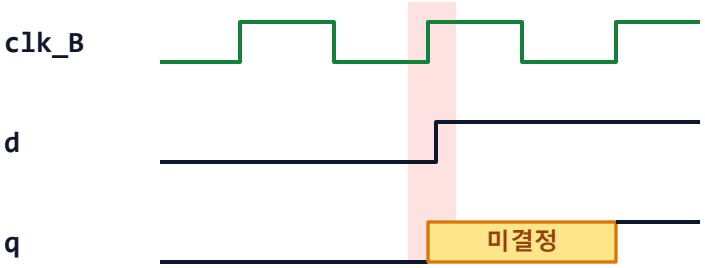
치명적인 이유

- 비동기 클럭 간 경로 → STA로 Setup · Hold 보장 불가
- metastable 값 → fan-out 로직마다 0 / 1 다르게 해석
- RTL 시뮬레이션에 metastability 없음 → 보드에서만 간헐적 오동작

CDC Fails as Metastability, Data Loss or Incoherence

① Metastability

Single-bit · 모든 CDC 경로

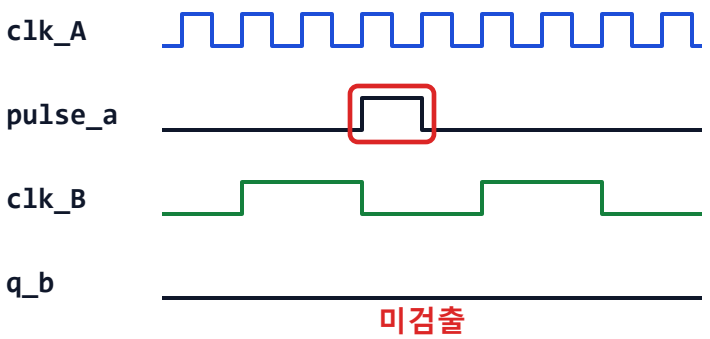


- d 변경 = clk_B setup · hold window 안
- q 출력이 일정 시간 0 / 1 미결정 후 임의 값 확정

해결 → 2-FF Synchronizer

② Data Loss

Single-bit pulse · fast → slow

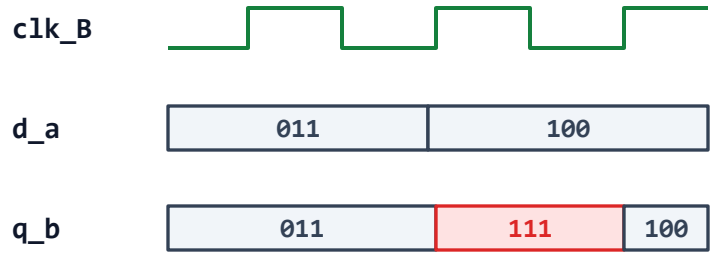


- pulse 폭 < clk_B 주기
- clk_B 엣지 사이에서 pulse 소멸 → event 소실

해결 → Toggle Synchronizer · Handshake

③ Incoherence

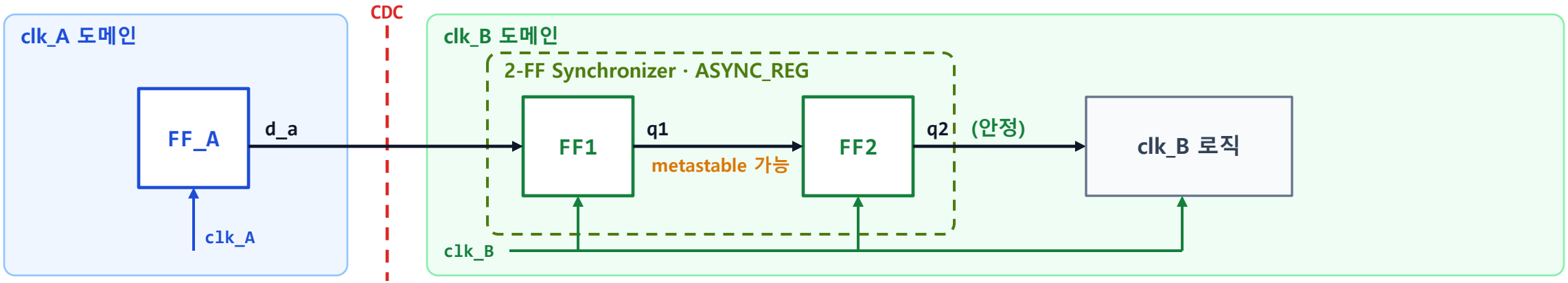
Multi-bit bus



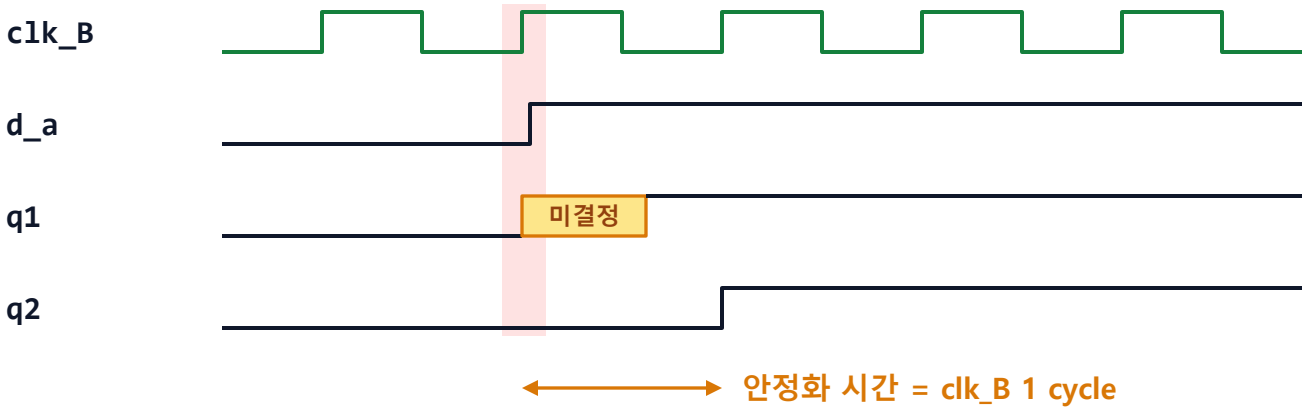
- bit별 metastability 해소 결과 상이
- 1 cycle 동안 존재하지 않는 값 수신 (011 → 111 → 100)

해결 → Gray Code · Handshake · Async FIFO

Single-Bit Level Signals Cross Through a 2-FF Synchronizer



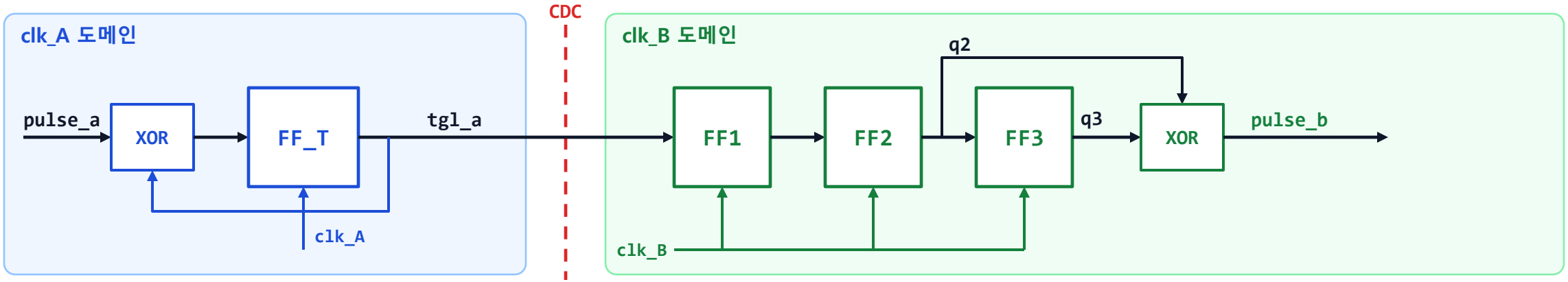
동작 파형

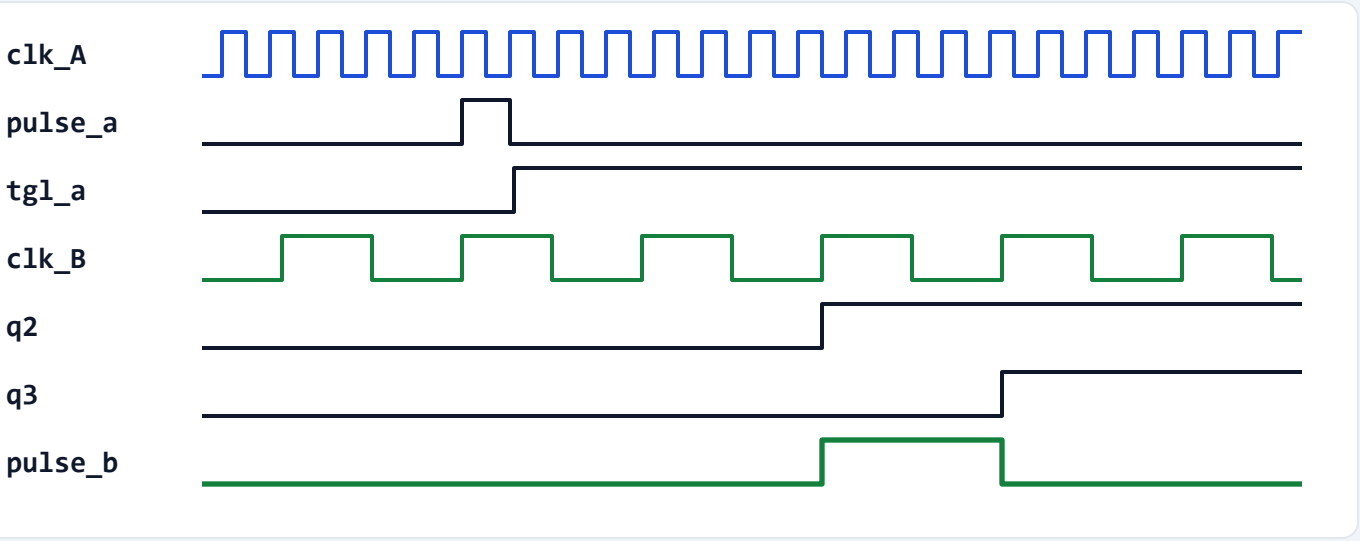


사용 조건

- source = FF 출력 직결, 조합 로직 금지 (Glitch 동기화 방지)
- d_a 유지 ≥ clk_B 1.5 cycle
- FF1 · FF2 인접 배치 (ASYNC_REG) → 안정화 시간 확보
- latency 2 cycle, 고속 clk_B → 3-FF로 MTBF 증가

Short Pulses Cross as a Toggle, Then Edge Detect Restores Them





동작 · 조건

- pulse → toggle level 변환, level만 2-FF 동기화
- q2 ^ q3 edge detect → clk_B 1 cycle pulse 복원
- pulse 간격 ≥ clk_B 3 cycle (이전 toggle 동기화 완료 필요)
- 짧은 간격 연속 event → Handshake · Async FIFO

Multi-Bit Data Crosses via Gray Code, Handshake or Async FIFO

03

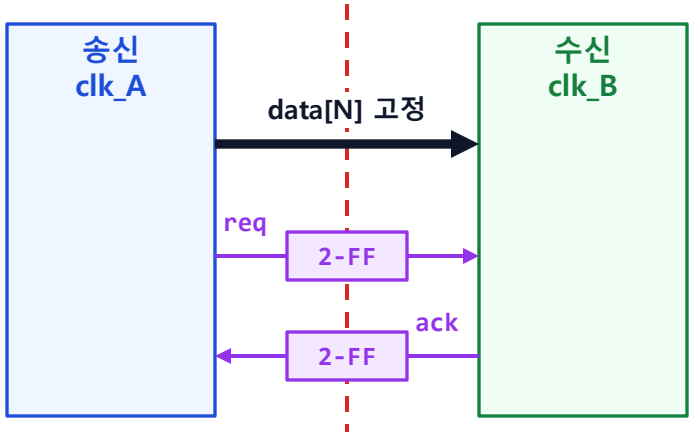
① Gray Code

값	Binary	Gray
3	011	010
4	100	110
변경 bit	3	1



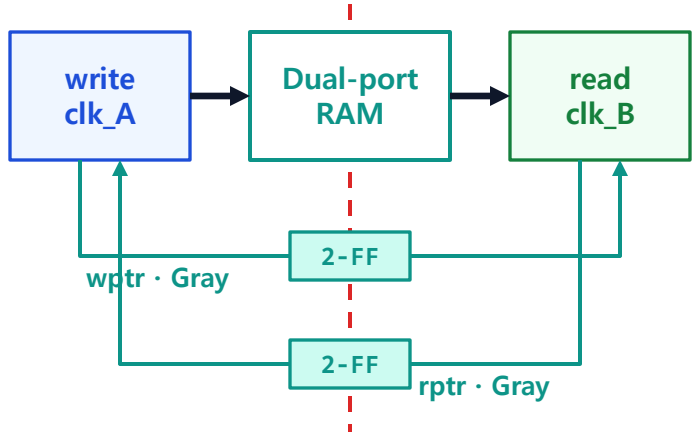
- 연속 값 사이 1 bit만 변경
- 동기화 오차 = 이전 값 또는 새 값만 발생
- 조건: +1 · −1씩 변하는 counter · pointer

② Handshake



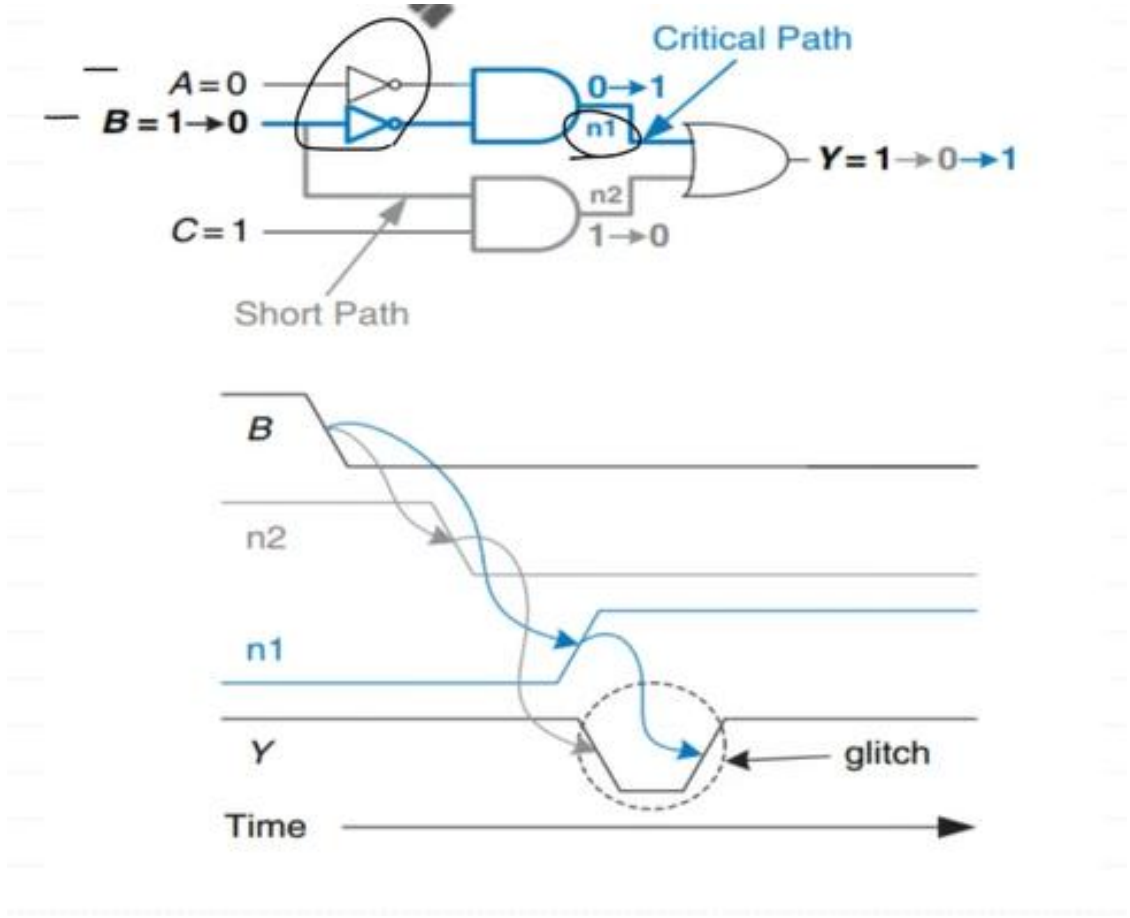
- data 고정 → req 1 bit만 동기화
- req 확인 시점 data 안정 → 수신 측 직접 캡처
- ack 회신 후 다음 data
- data bus 동기화 없음, word당 수 cycle 소요

③ Async FIFO



- data = dual-clock RAM 저장, 동기화 없음
- pointer만 Gray 변환 후 2-FF 동기화
- empty = 수신 측 판정, full = 송신 측 판정
- 연속 data stream 전달

Unequal Path Delays Create a Glitch



B : 1 → 0, A = 0, C = 1

- Short Path (n2): B 직접 → AND → 빠르게 0
- Critical Path (n1): B → inverter → AND → 늦게 1
- n1 · n2 둘 다 0인 구간 발생 → Y = 1 → 0 → 1
- Glitch 폭 = 두 경로 delay 차이

발생 위치

- 조합회로 출력 (입력 간 도착 시간 차이)
- FF 출력 = 클럭 엣지에서만 변경 → Glitch 없음

Register Outputs Only Where Glitches Matter

Combinational vs Sequential Output

항목	CL 출력	SL 출력
Glitch	**통과**	**제거**
응답	**전파 delay만**	**+1 cycle 지연**
면적 · Cost	FF 없음	FF 수 증가
전력 · 발열	클럭 부하 없음	FF 클럭 toggle 전력 증가
Timing	Critical Path 길어짐	경로 분할 → 안정

FF 필요

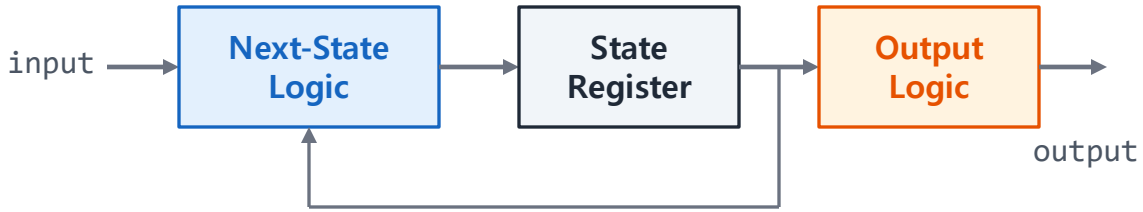
- clock · enable · async reset으로 쓰는 신호
- 다른 클럭 도메인 · 외부 칩 입력
- Critical Path 분할 필요 구간

FF 불필요

- 같은 클럭 FF가 엣지에서만 읽는 내부 신호
- 다음 엣지 Setup 전 안정 → Glitch 무관
- LED처럼 ns 단위 Glitch가 의미 없는 출력

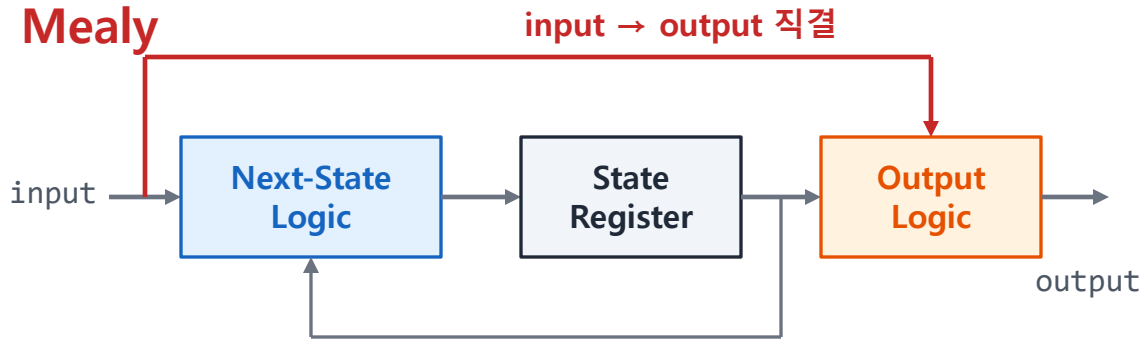
Moore Output Depends on State Only

Moore



`output = f(state)`

Mealy

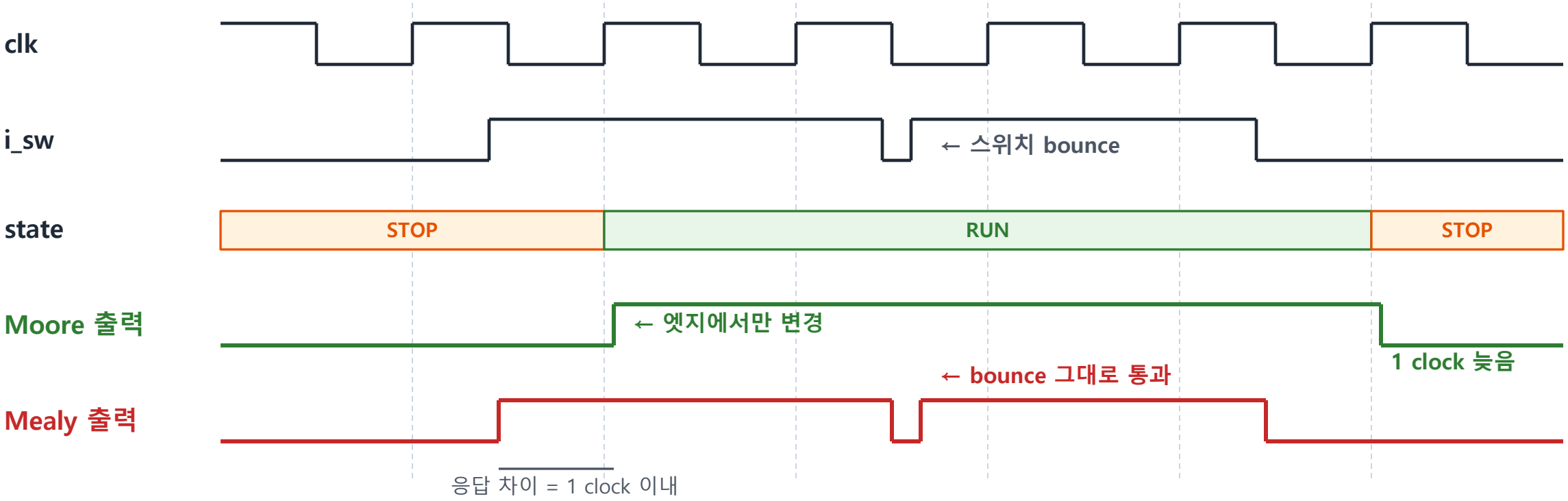


`output = f(state, input)`

항목	Moore	Mealy
출력 결정	현재 state만	현재 state + 현재 input
출력 변경 시점	클럭 엣지 이후 (state 변경 시)	input 변경 즉시 (같은 cycle)
응답 지연	1 clock	0 clock
input Glitch · bounce	**출력에 전달 안 됨**	**출력에 그대로 전달**
필요 상태 수	같은 기능 기준 많거나 같음	적거나 같음
Timing path	register → output	input → output 조합 경로 추가

Mealy Reacts Instantly, Moore Waits One Clock

05



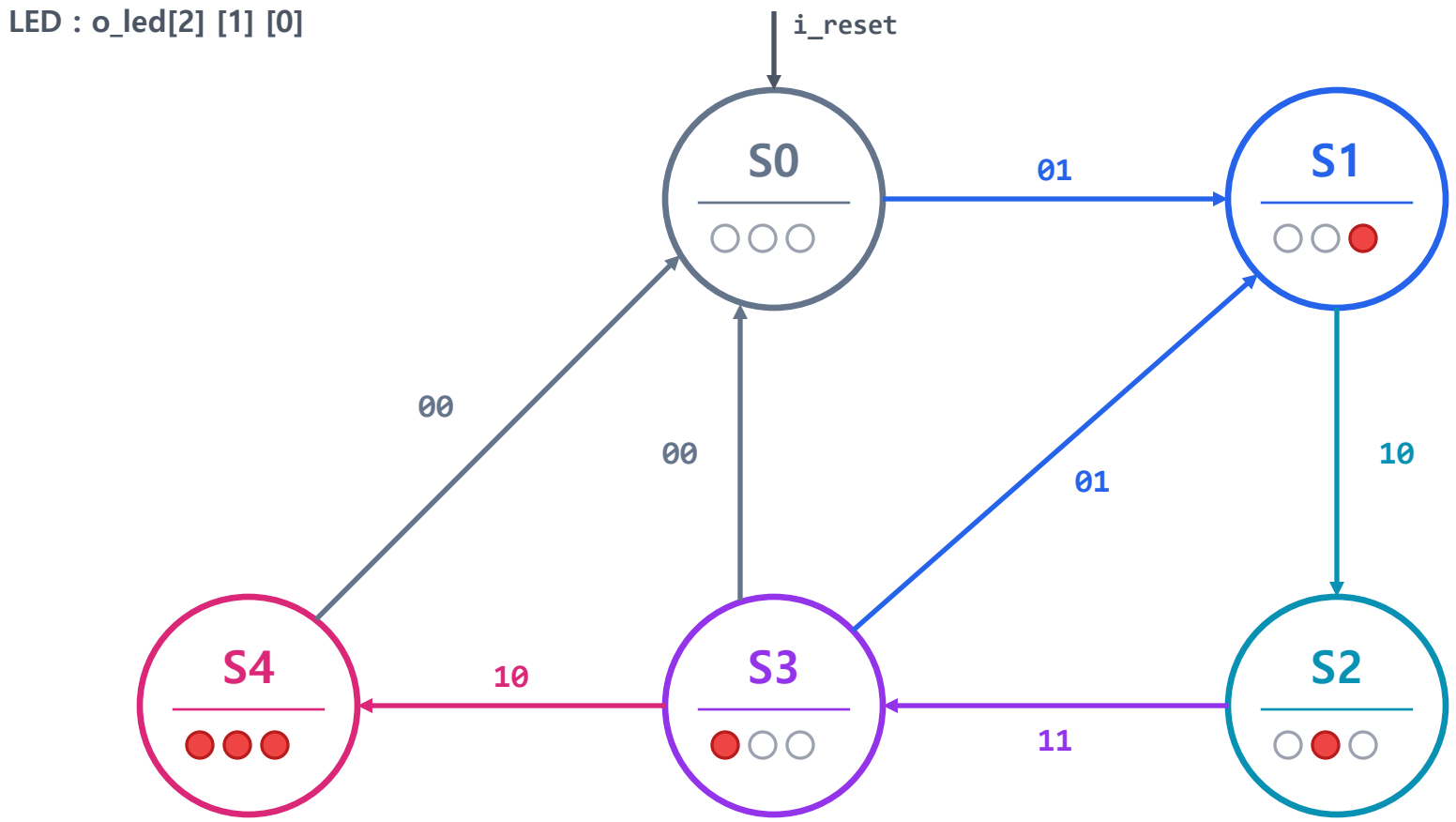
Moore

- 클럭 엣지에서만 출력 변경, 엣지 사이 input 변화 무시
- 대가: input 대비 최대 1 clock 지연

Mealy

- input 변경 즉시 출력 반영, 지연 없음
- 대가: input의 Glitch · bounce가 출력까지 전달

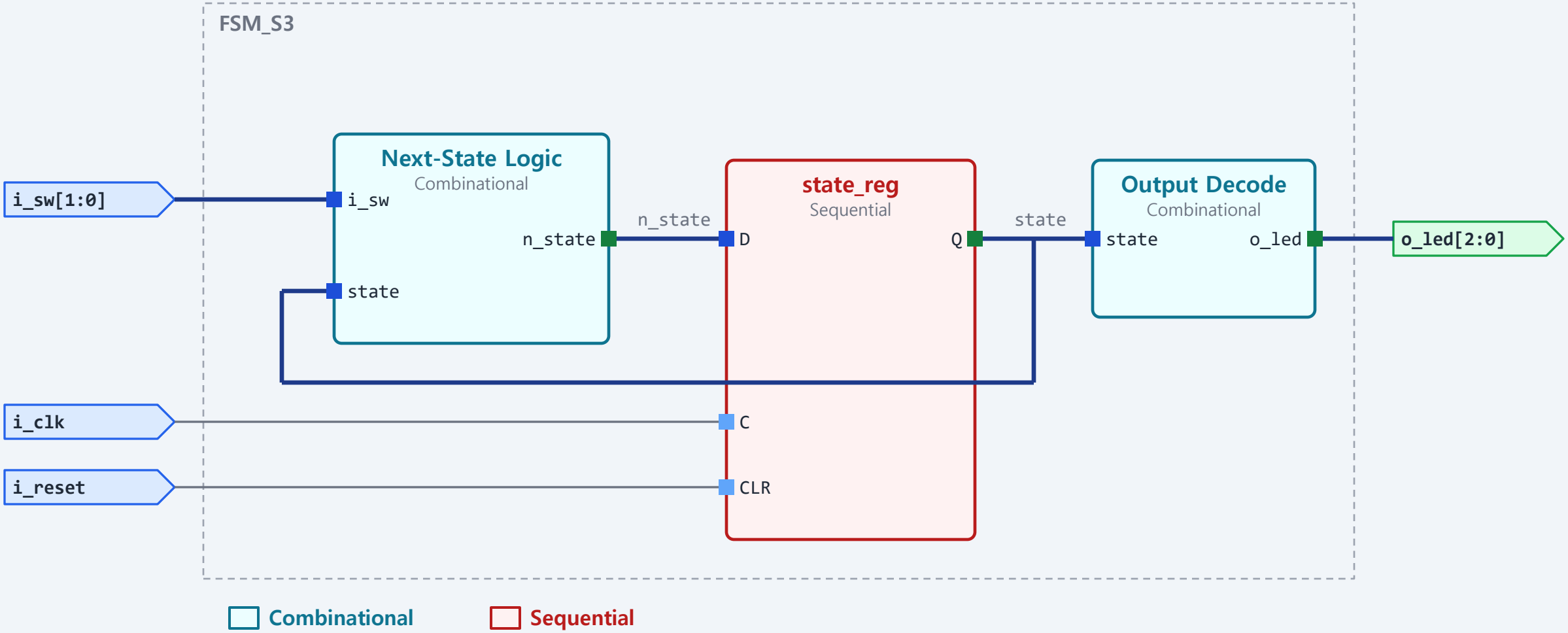
Five States Step the LEDs by Switch Code



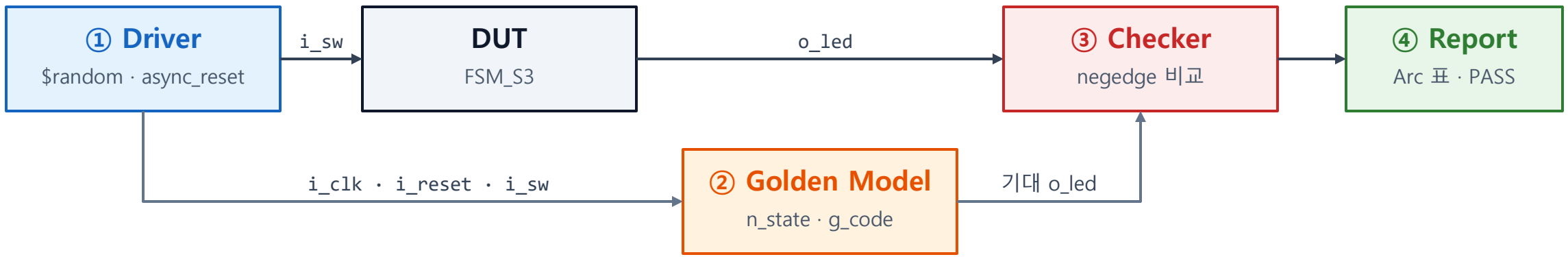
HOLD 조건

State	HOLD	o_led
S0	≠ 01	000
S1	≠ 10	001
S2	≠ 11	010
S3	11	100
S4	≠ 00	111

LED Decode Reads Only the State Register



TB Compares o_led with a Golden Model Every Cycle



① Driver

- negedge + 1 ns에 i_sw 인가
- $random 하위 2 bit → 00 · 01 · 10 · 11
- 시작 시 async_reset 1회 → S0

② Golden Model

- TB 전용 integer state
- n_state function = 다음 state
- g_code function = 기대 o_led
- DUT와 같은 posedge · async reset으로 갱신

③ Checker

- 매 negedge, i_reset = 0 구간만 비교
- o_led !== g_code(state) → err_c1 + 1
- 시각 · sw · 기대값 · 실제값 출력

④ Report

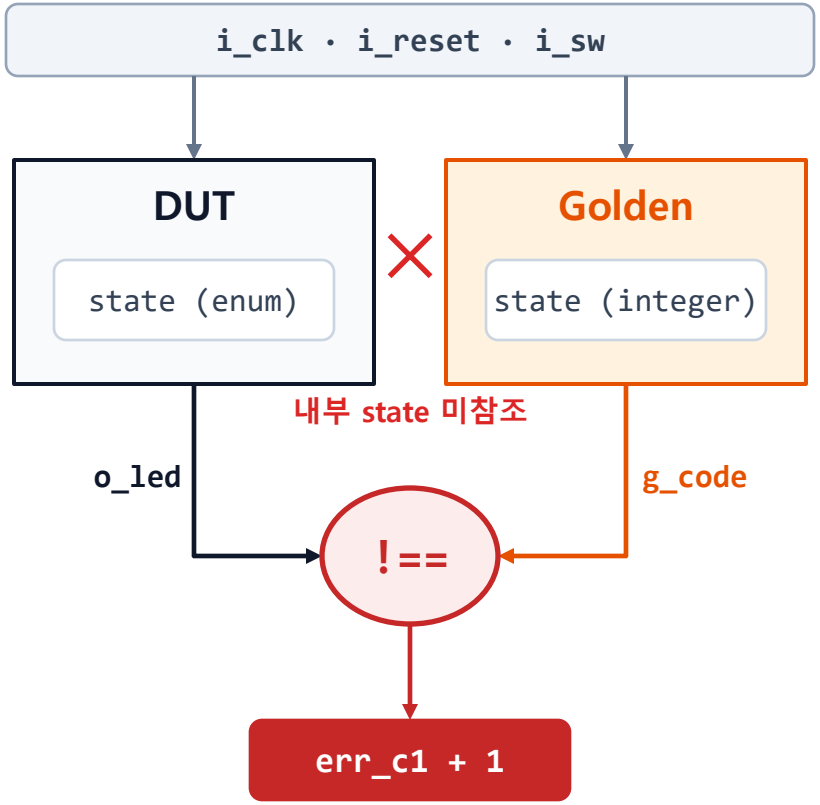
- cycle 10000 도달 시 종료
- Arc coverage 표 (state × i_sw)
- err_c1 = 0 → PASS 출력

Golden Model Keeps Its Own State and Output Table

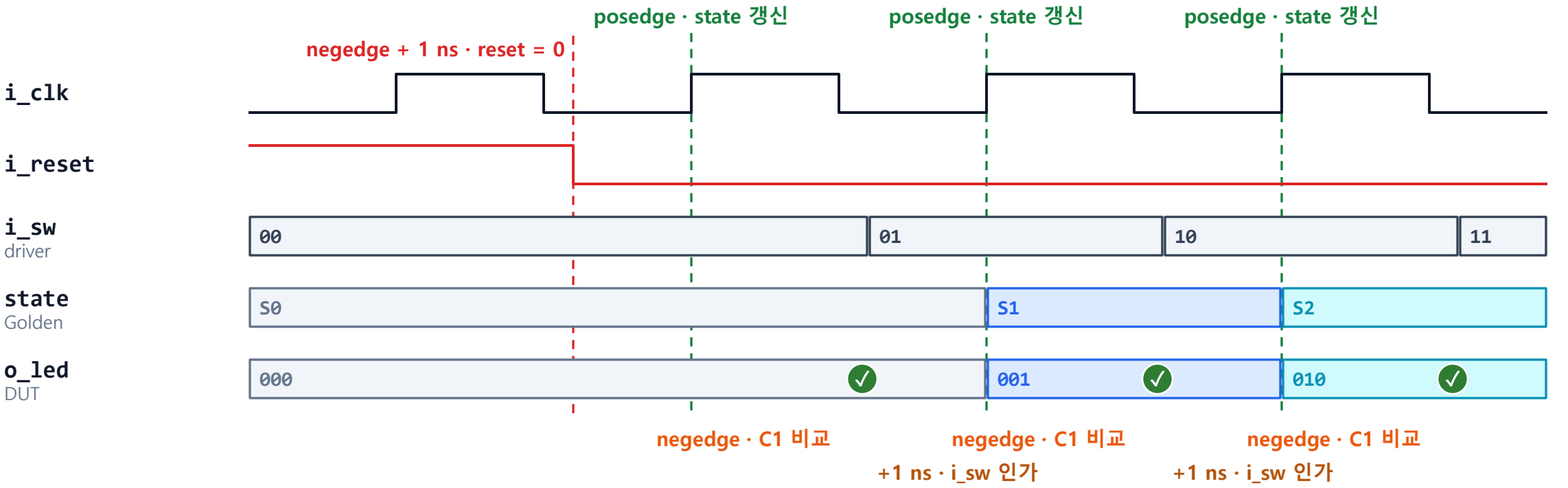
DUT vs Golden Model

항목	DUT · FSM_S3	Golden · tb_FSM_S3
state 변수	state_t enum	integer state
state 번호	enum S0 ~ S4	localparam S0 ~ S4 = 0 ~ 4
Next-State	always @(*) · case + if-else	function n_state · 삼항 연산
미사용 code	default → n_state = state	default → S0
출력	always @(*) · case → o_led	function g_code(state)
갱신 시점	posedge i_clk · posedge i_reset	동일 엣지 · 동일 reset

분리 구조



Driver Changes i_sw 1 ns After Each Negedge



구동 시점

- negedge + 1 ns · i_sw 인가
- 다음 posedge까지 4 ns 안정

입력값

- $random 하위 2 bit · 매 cycle 새 값
- 시작 시 async_reset 1회 → S0

판정 시점

- 매 negedge · i_reset = 0 구간
- C1 비교 후 i_sw 변경 → race 없음

Random Input Covers All 20 Arcs in 10,000 Cycles

커버리지 확보 방법

① $random → i_sw[1:0]
매 cycle 새 값 · 4가지 code

② posedge · cov[state][i_sw] + 1
현재 state × 입력 = arc 1회 기록

③ cycle 10000 도달까지 반복
종료 시 5 × 4 arc 표 출력

S4 도달 경로



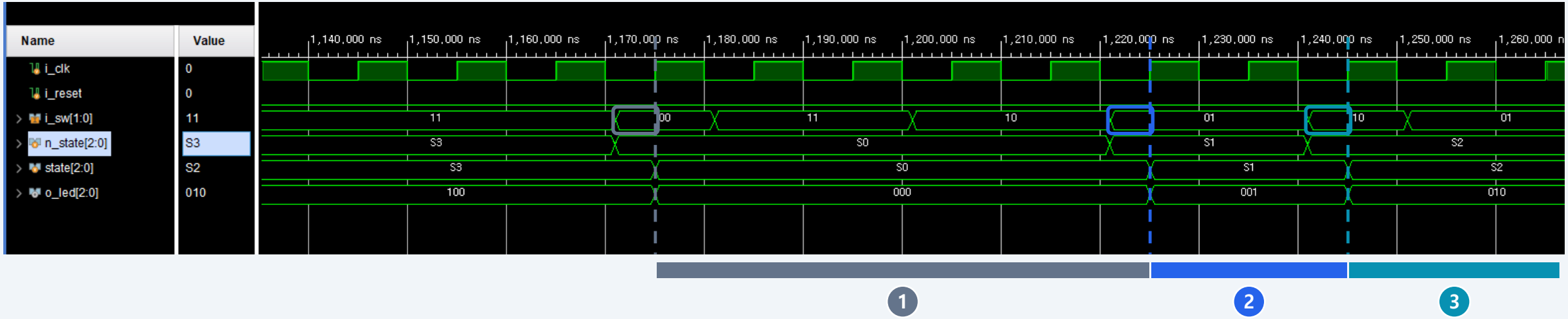
Arc Coverage (state × i_sw, 실행 횟수)

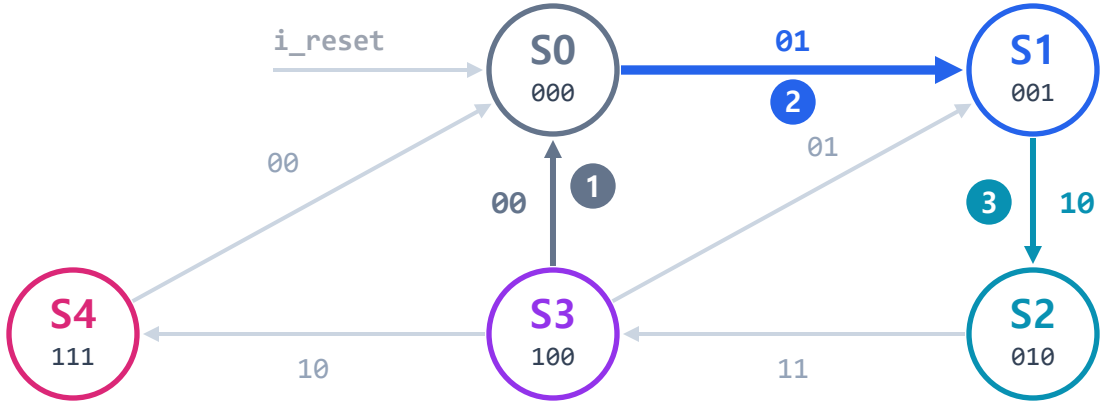
state	00	01	10	11
S0	499	490	489	514
S1	783	734	767	754
S2	770	793	724	767
S3	258	277	232	254
S4	232	236	220	207

Tcl Console · tb_FSM_S3

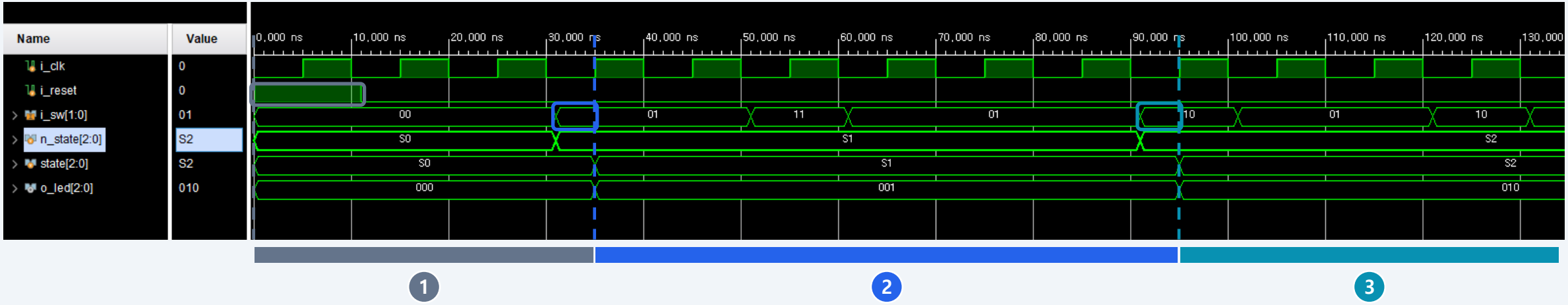
```
-----------------------------------------------
 Arc coverage (state x sw)   00   01   10   11
   S0                       499  490  489  514
   S1                       783  734  767  754
   S2                       770  793  724  767
   S3                       258  277  232  254
   S4                       232  236  220  207
[PASS] 'o_led == Golden Value' for all cases
```

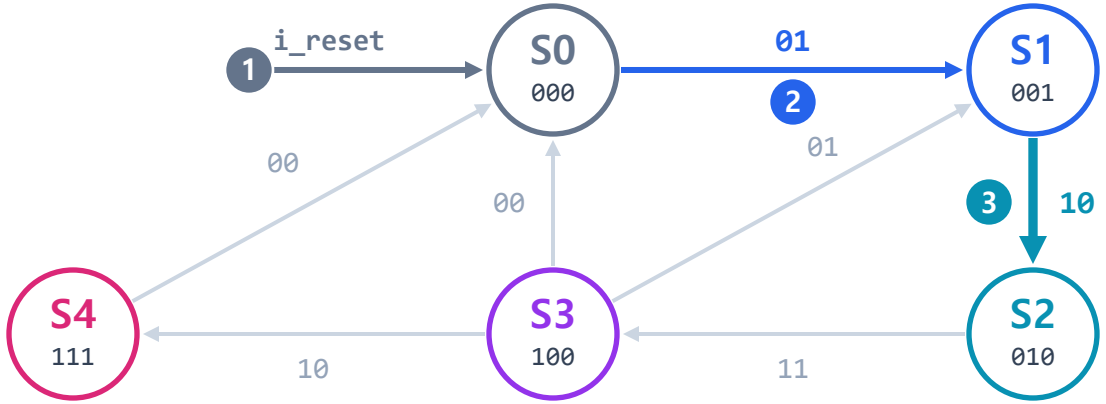
S0 Moves to S1 Only on 01



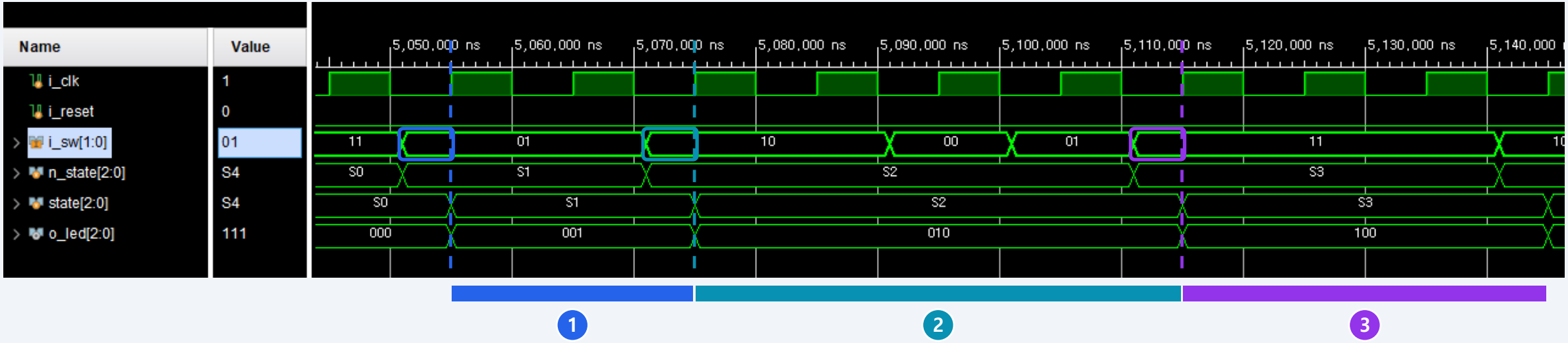


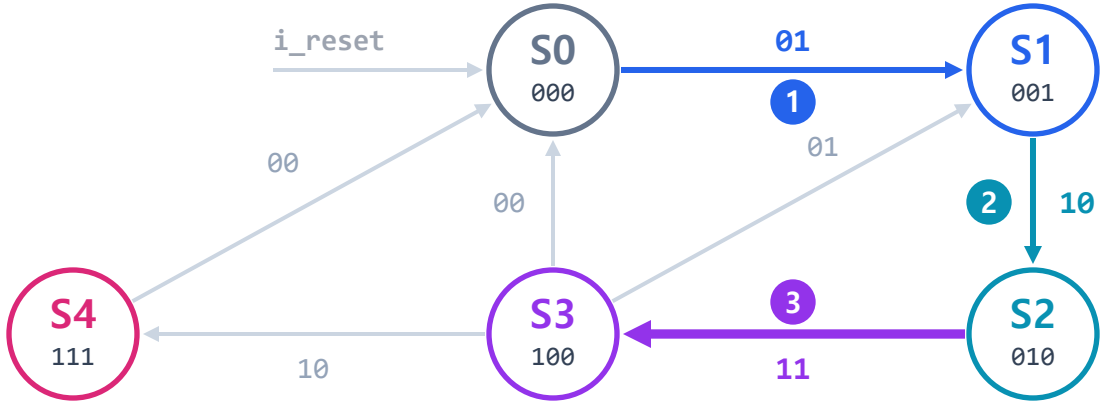
S1 Moves to S2 Only on 10



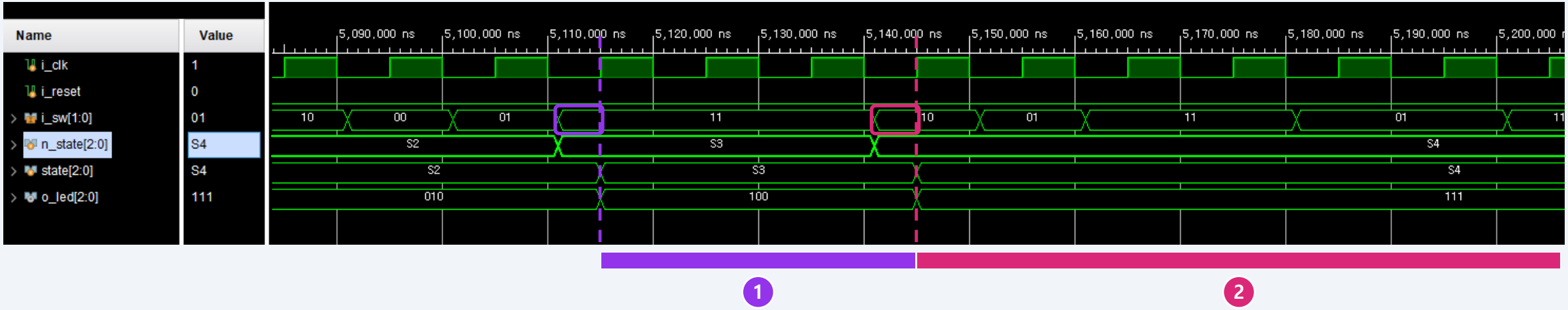


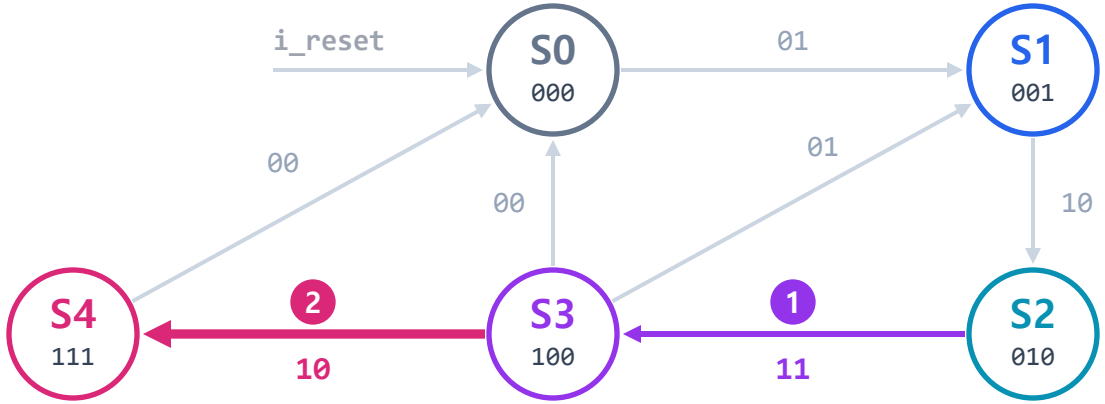
S2 Moves to S3 Only on 11



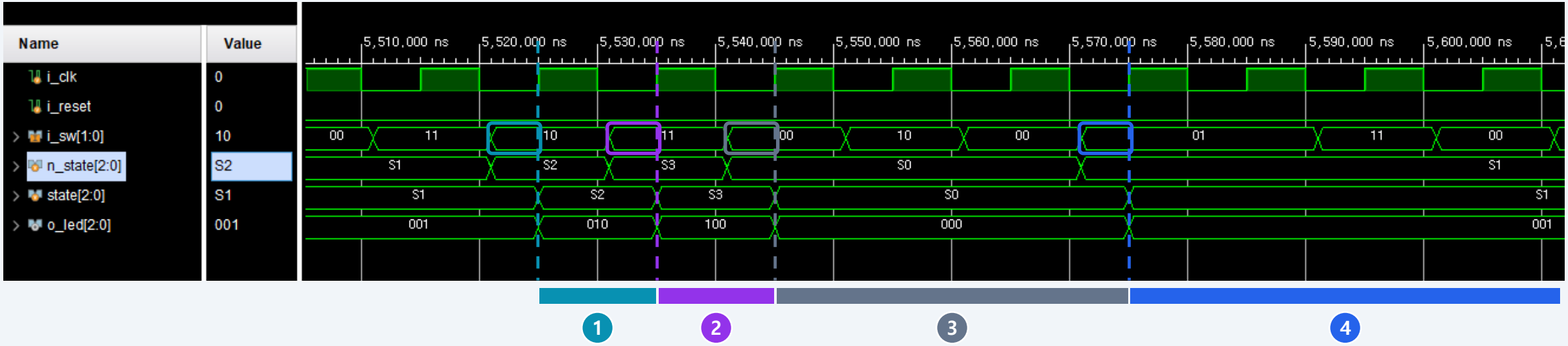


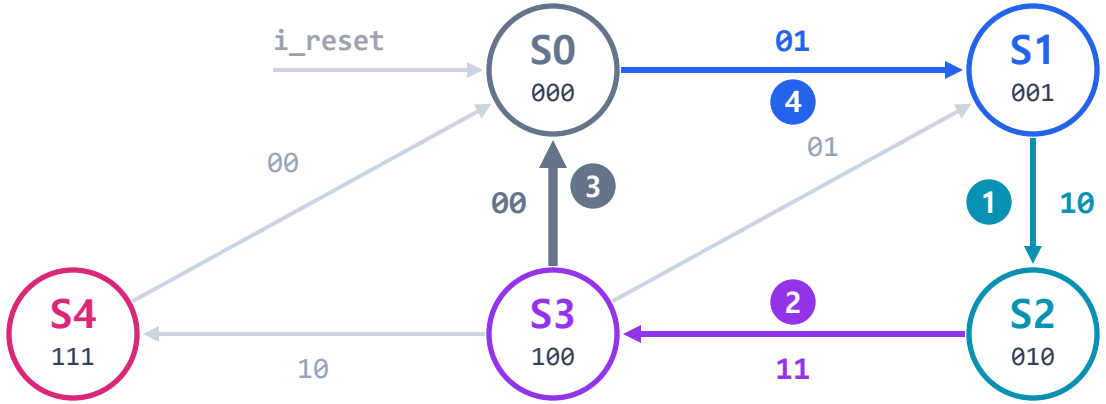
S3 Moves to S4 on 10



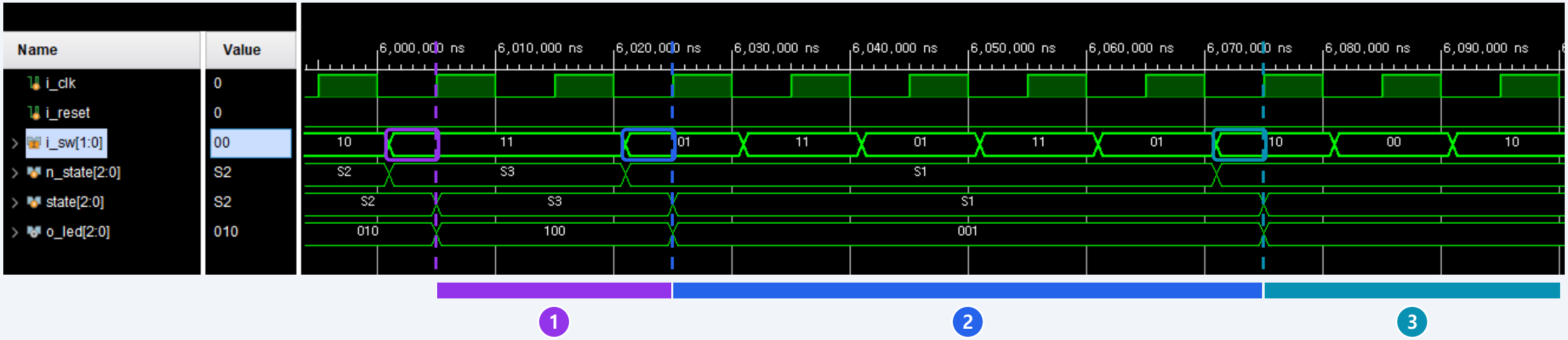


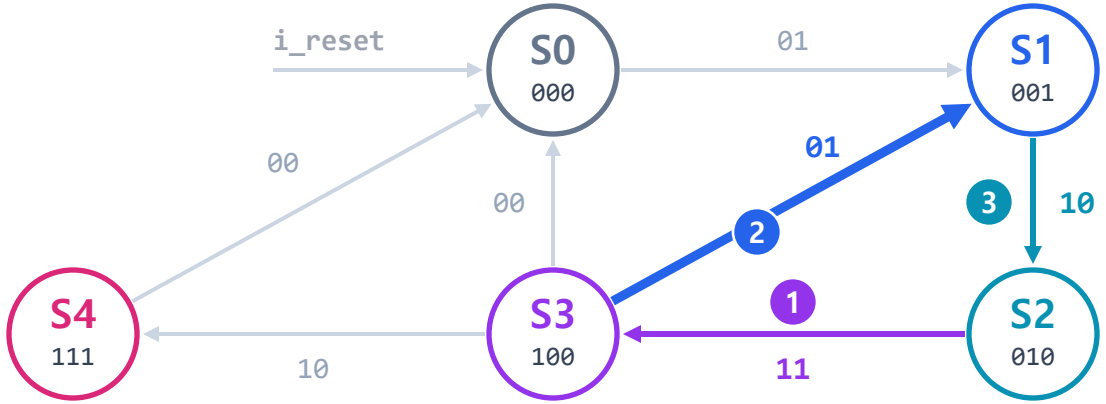
S3 Returns to S0 on 00



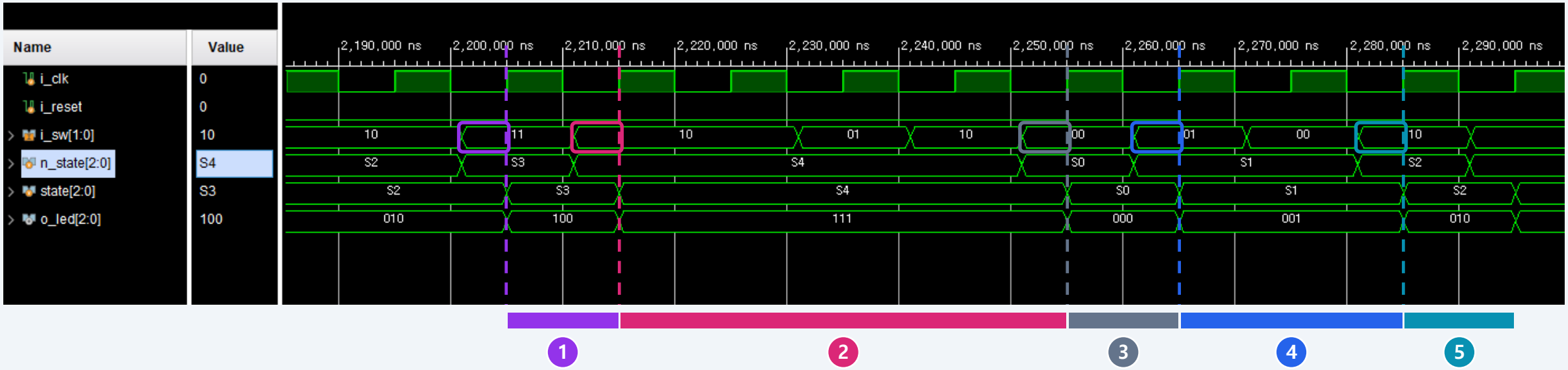


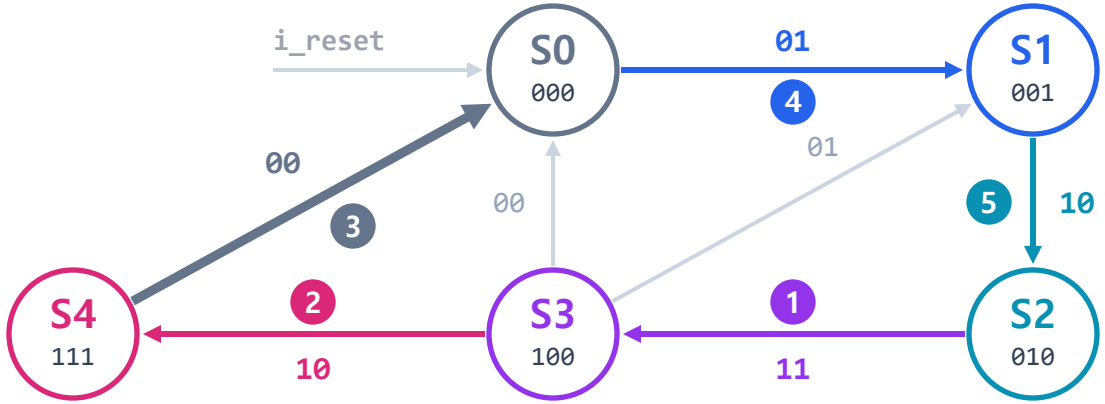
S3 Returns to S1 on 01





S4 Returns to S0 Only on 00



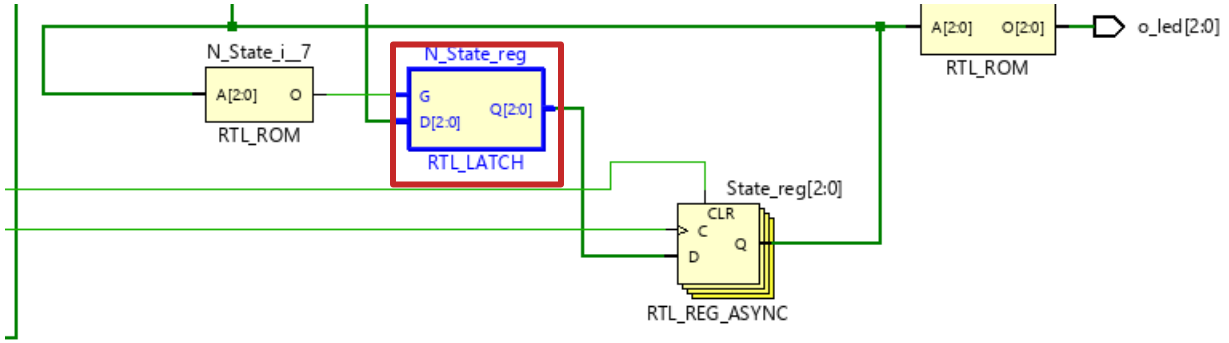


Missing Default Leaves N_State Unassigned, Inferring a Latch

State[2:0] = 8가지 코드

000 S0	001 S1	010 S2	011 S3
100 S4	101 대입 없음	110 대입 없음	111 대입 없음

default 없음 → RTL_LATCH

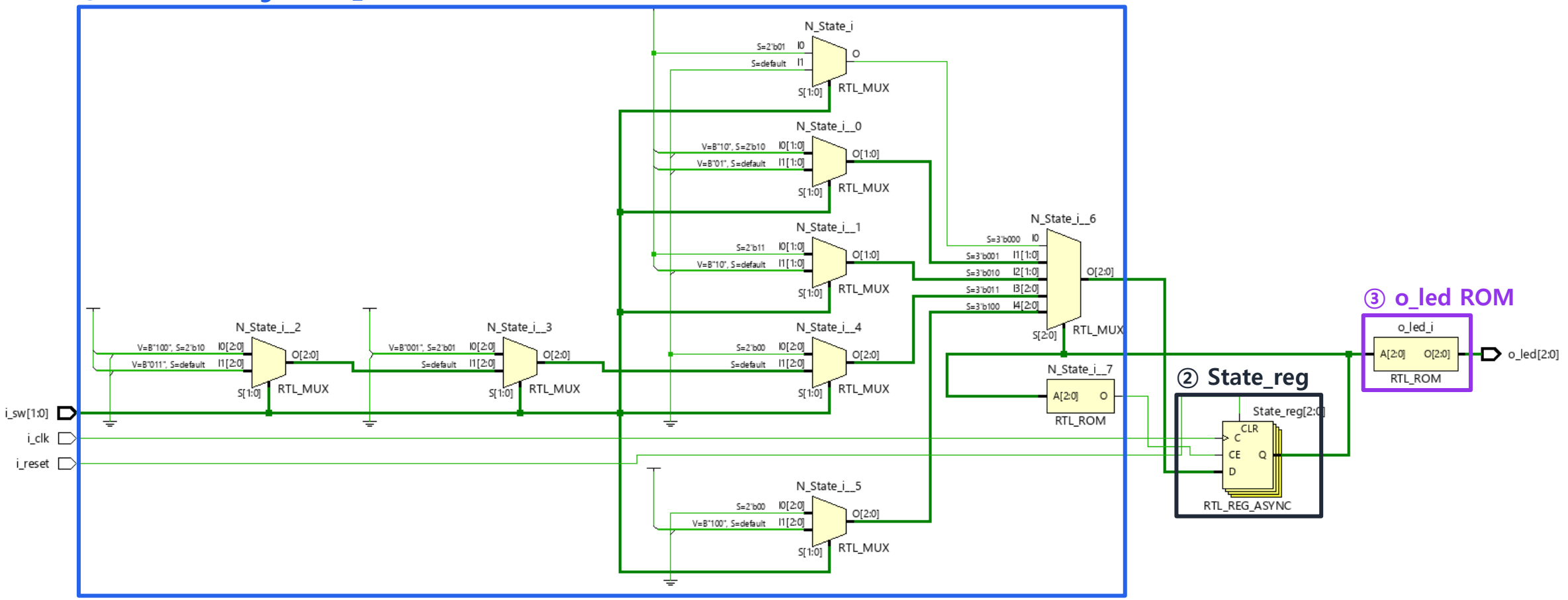


결과

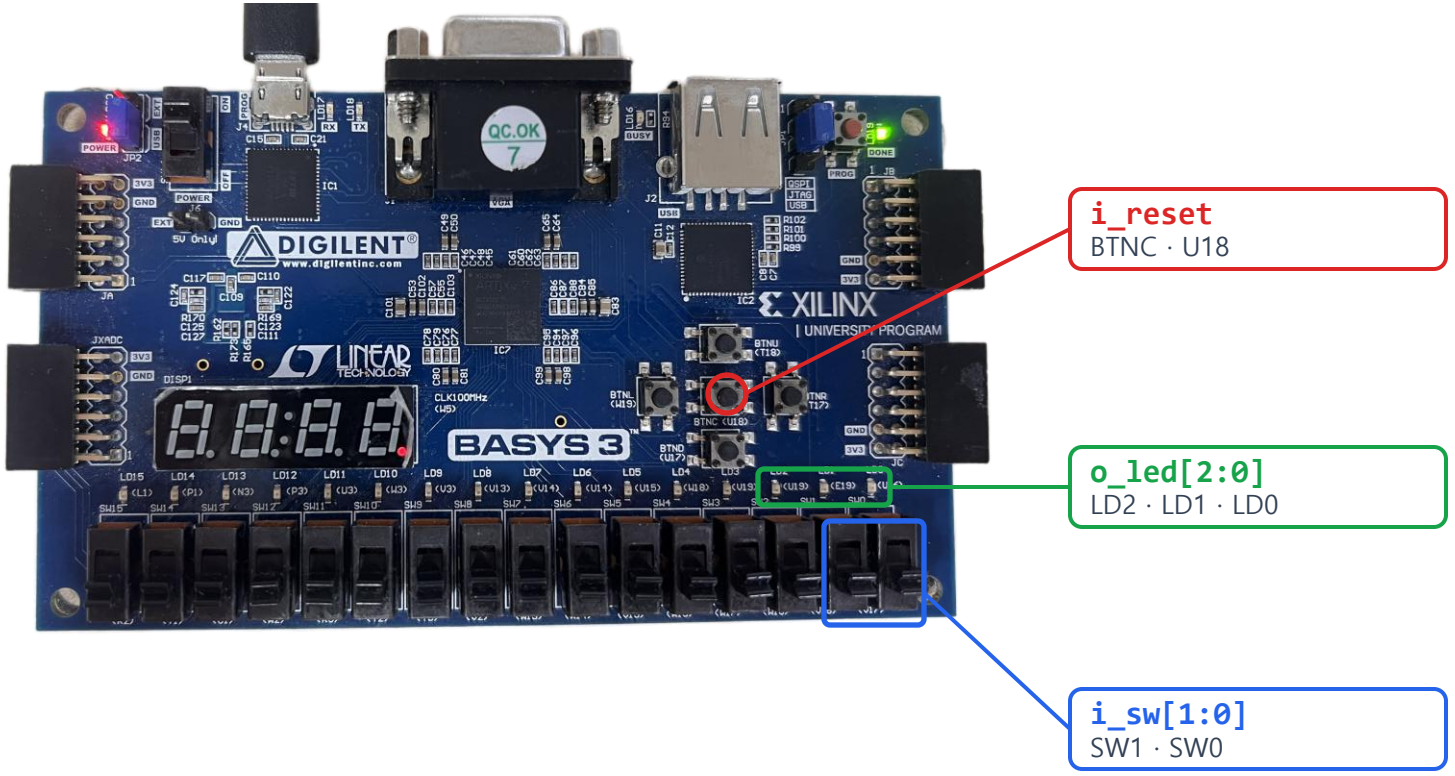
Case 문에서 default문을 명시하지 않을 경우 합성단계에서 Latch가 발생

With Default, Only the State Register Stores

① Next-State Logic · RTL_MUX



SW1 · SW0 Drive i_sw, BTNC Resets, LD2 ~ LD0 Show o_led

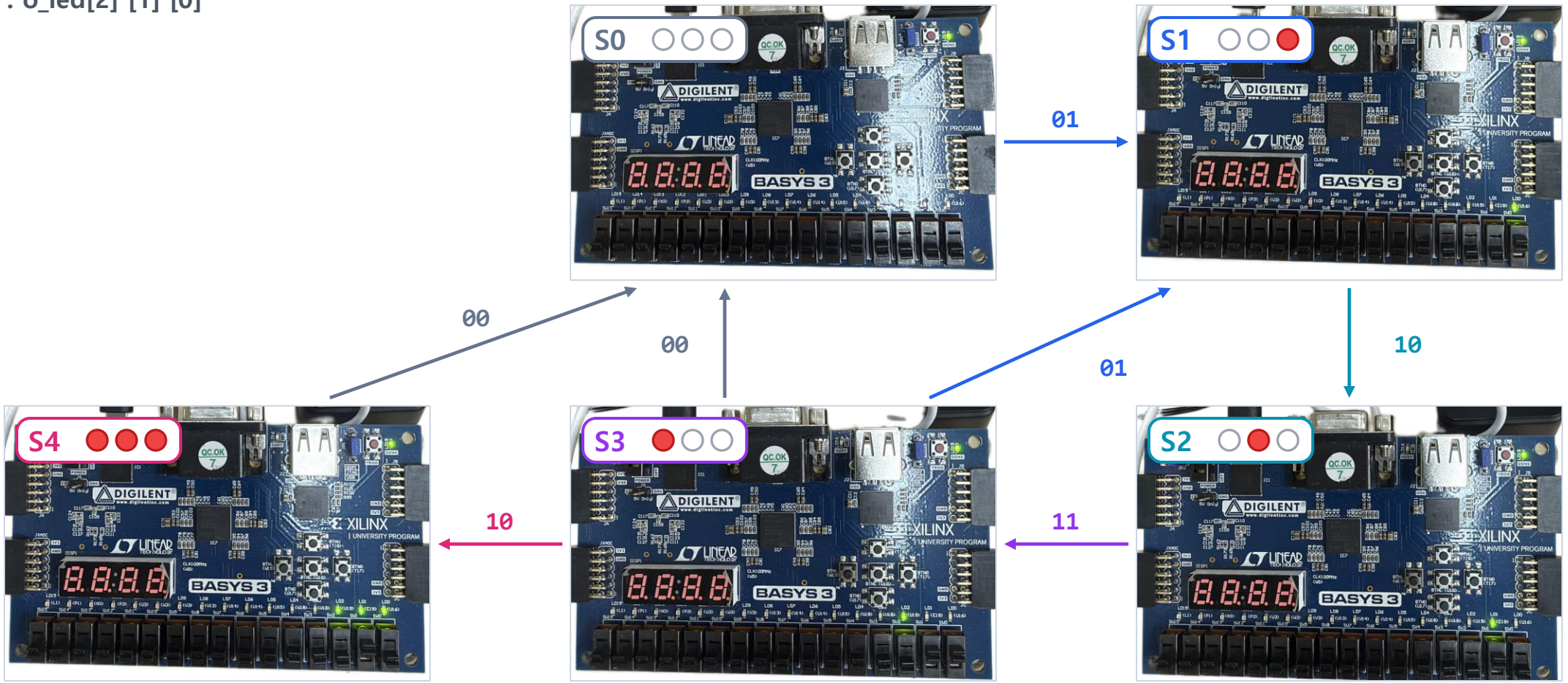


FSM_CON.xdc

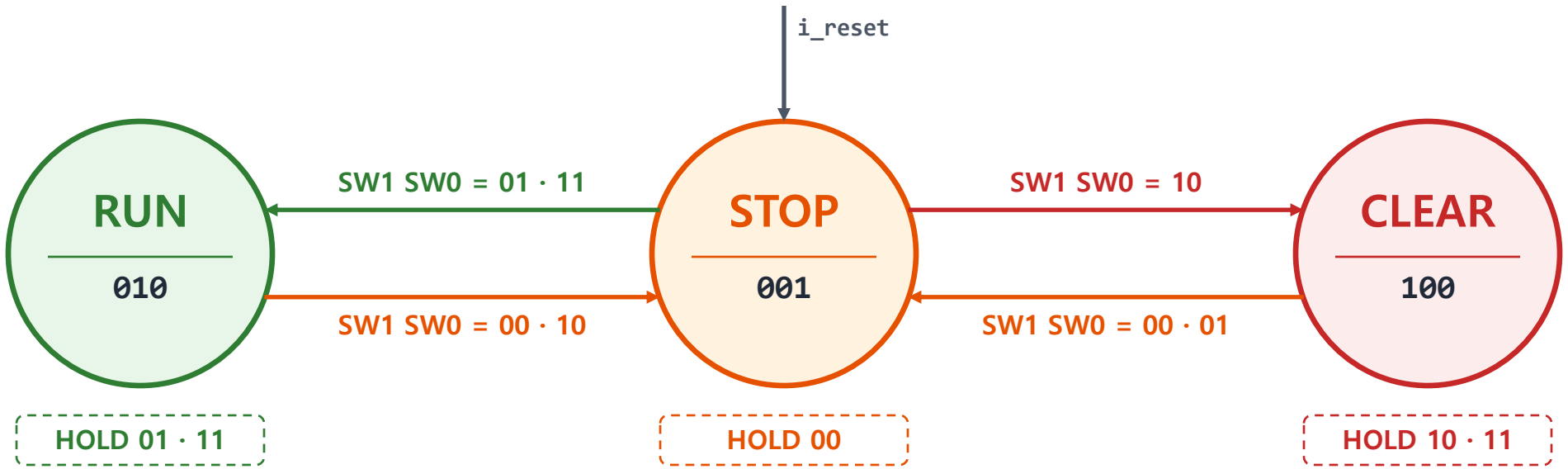
Port	Dir	Pin	Board
i_clk	input	W5	100 MHz
i_reset	input	U18	BTNC
i_sw[0]	input	V17	SW0
i_sw[1]	input	V16	SW1
o_led[0]	output	U16	LD0
o_led[1]	output	E19	LD1
o_led[2]	output	U19	LD2

Board LEDs Match All Five State Codes

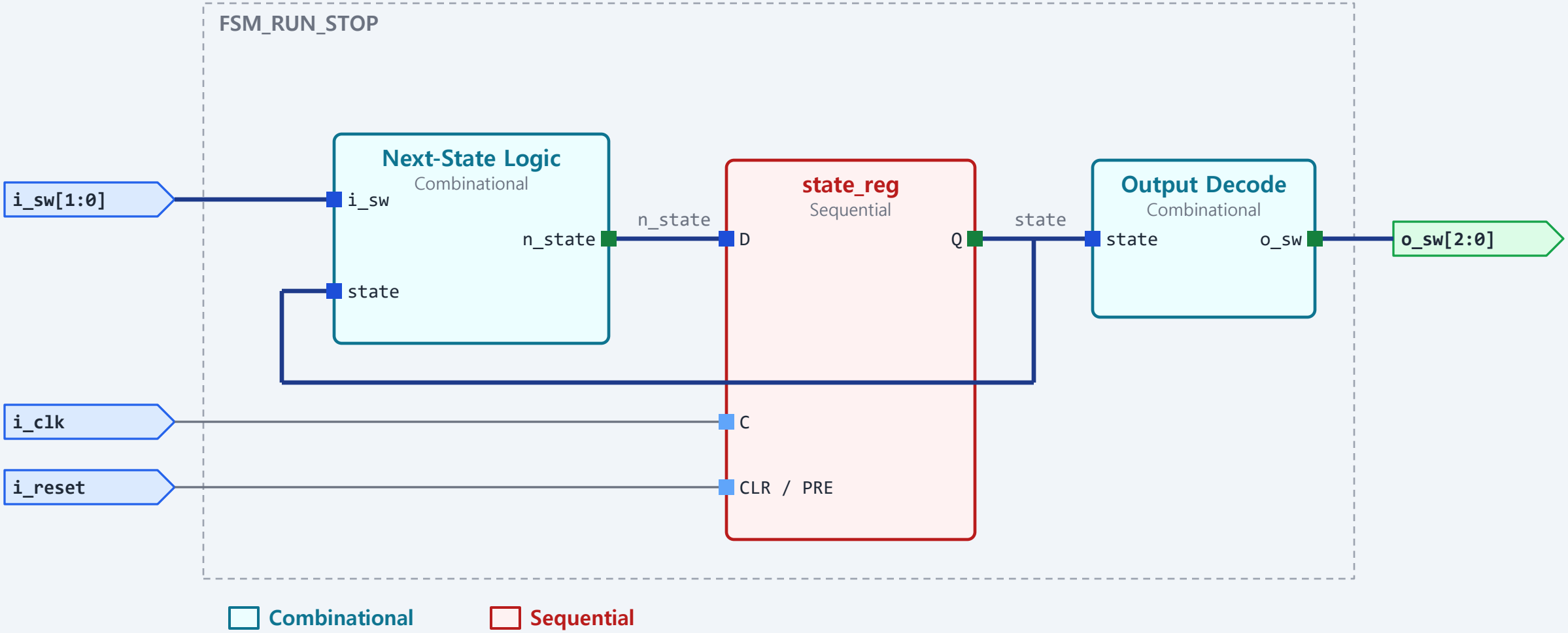
LED : o_led[2] [1] [0]



STOP Is the Hub Between RUN and CLEAR



Combinational Logic Wraps One State Register

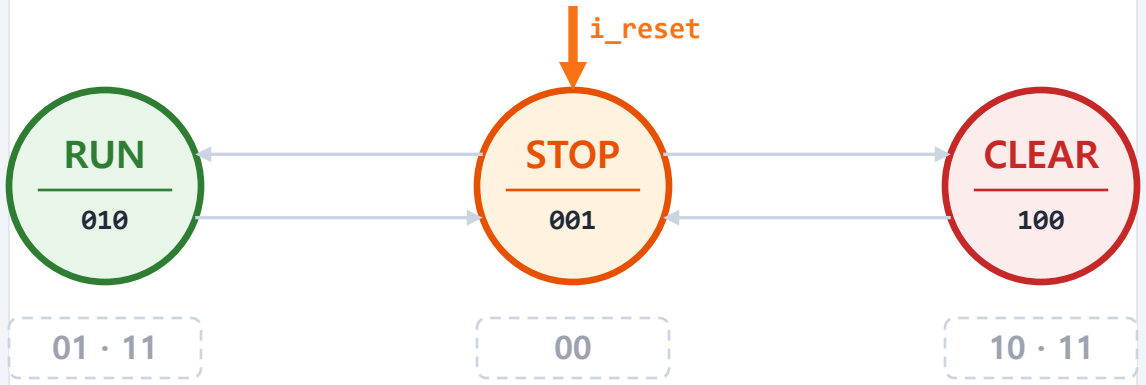


Enum and State Register Hold the Current State


```
typedef enum logic [2:0] {
    clear = 3'd0, stop = 3'd1, run = 3'd2
} state_t;
state_t state, n_state;

always @(posedge i_clk or posedge i_reset) begin
    if (i_reset) state <= stop;
    else         state <= n_state;
end
```

i_reset → STOP (클럭 무관)

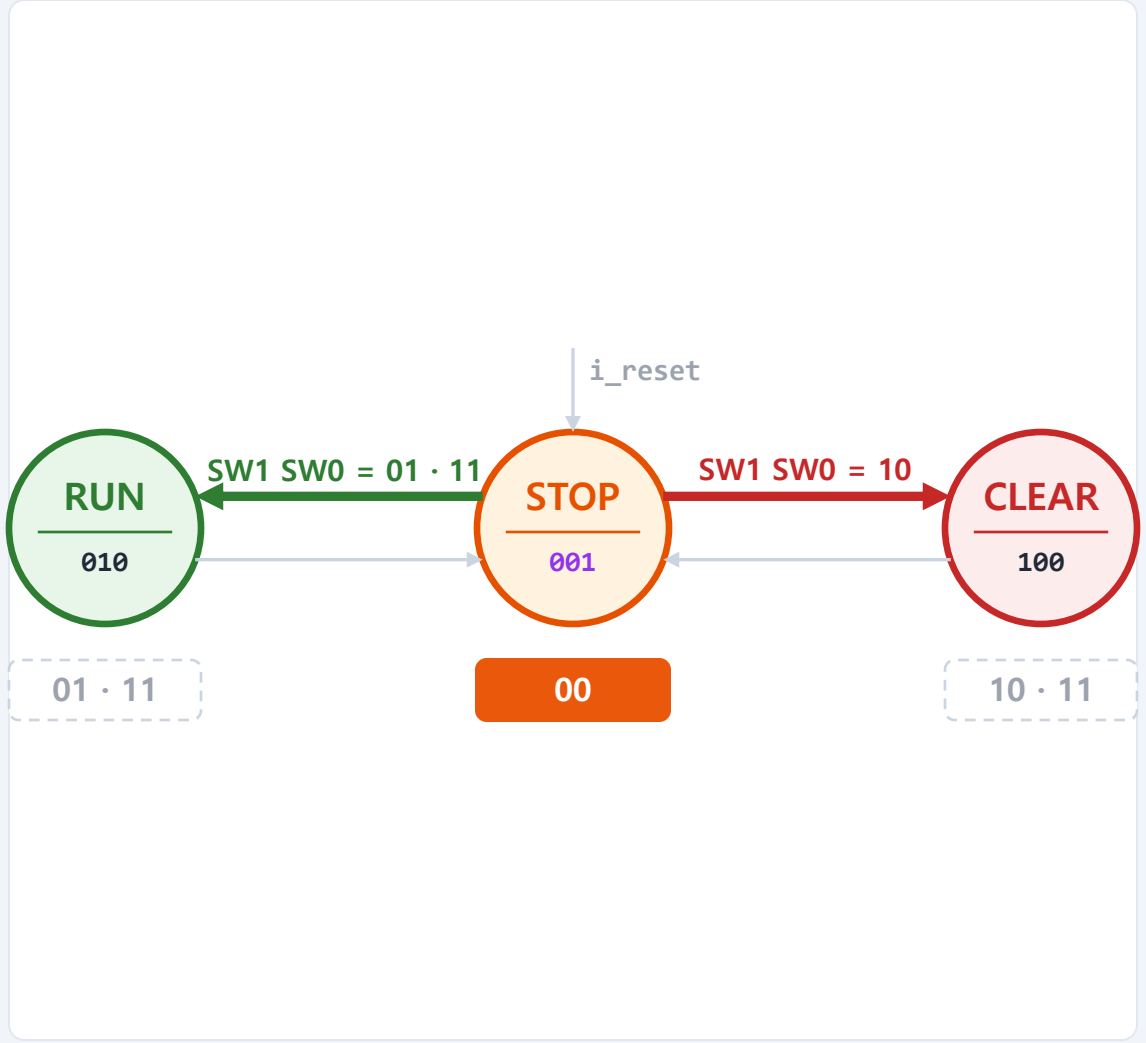


상승 엣지마다 state ← n_state

CLEAR = 0 | STOP = 1 | RUN = 2

STOP Branch Selects RUN or CLEAR


```
case (state)
  stop : begin
    case (i_sw)
      2'b11   : n_state = run;
      2'b01   : n_state = run;
      2'b10   : n_state = clear;
      default : n_state = stop;
    endcase
    o_sw = 3'b001;
  end
```



RUN and CLEAR Branches Return to STOP


```
clear : begin
  if (!i_sw[1]) n_state = stop;
  else          n_state = clear;
  o_sw = 3'b100;
end
run : begin
  if (!i_sw[0]) n_state = stop;
  else          n_state = run;
  o_sw = 3'b010;
end
```



Default Branch Covers Unused Codes, No Latch

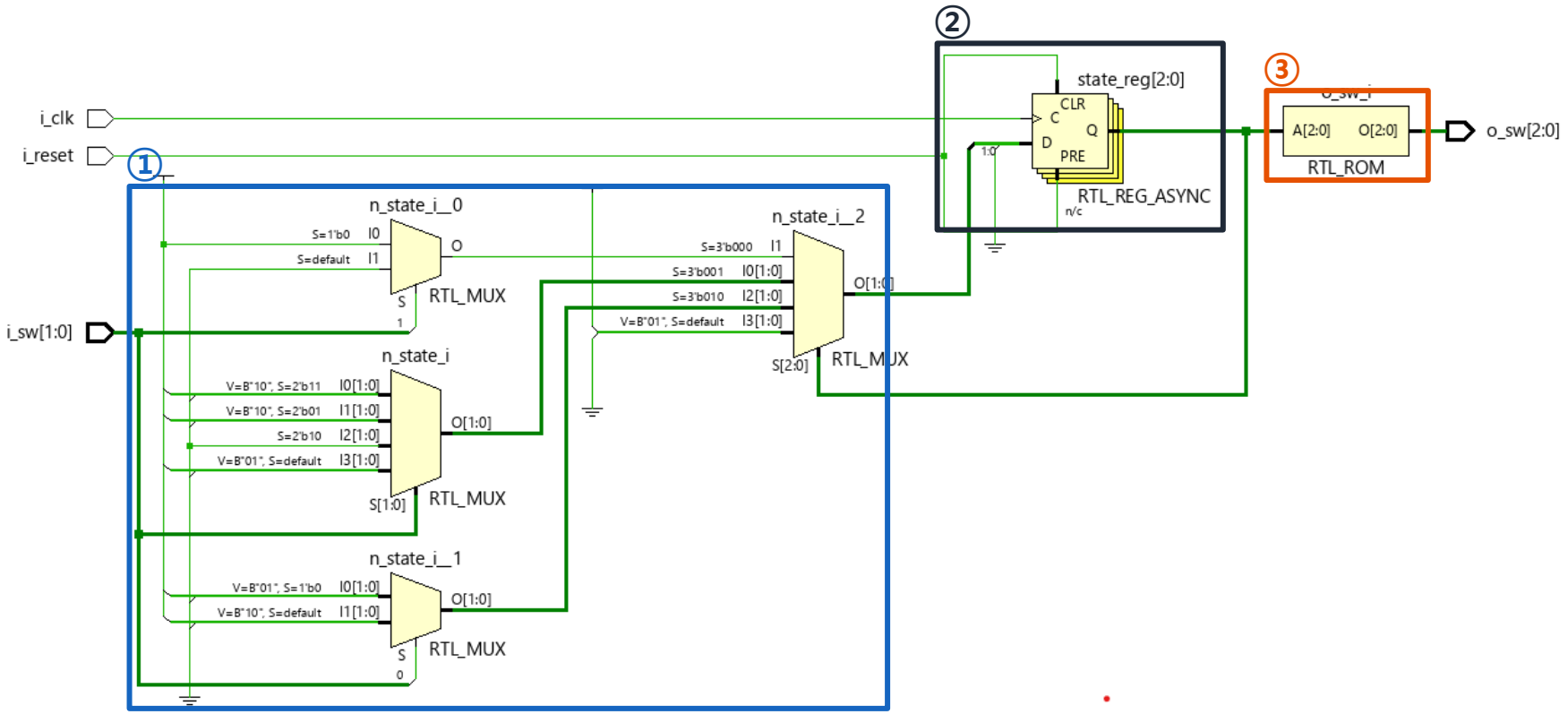

```
    default : begin
      n_state = stop;
      o_sw    = 3'b001;
    end
  endcase
```

state[2:0] 코드

000 CLEAR	001 STOP	010 RUN	011 → STOP
100 → STOP	101 → STOP	110 → STOP	111 → STOP

- 미사용 코드 5개 → default → STOP 복귀
- 모든 branch에서 n_state · o_sw 대입 → Latch 없음

Schematic Shows Output Decoded from State Only



① Next-State Logic

RTL_MUX × 4

i_sw · state → n_state

② State Register

RTL_REG_ASYNC

CLR · PRE = reset 시 001 (STOP) 로드

③ Output Decode

RTL_ROM

입력 A[2:0] = state만 연결, i_sw 연결 없음

Existing TB Walks Three Start States

tb_FSM_RUN_STOP 구성

- i_sw 변경 = negedge 시점 → 상승 엣지에서 input 안정
- set_sw: 한 값을 2 clock 유지 (RUN → STOP → CLEAR 최장 경로 2 천이 대응)
- phase 변수 k + reset으로 시작 state 구분, 판정 = 파형 육안 확인

k = 0 STOP 시작

`i_sw : 00 → 00 · 01 · 10 · 11`

STOP
→ RUN
→ STOP
→ CLEAR
→ CLEAR

k = 1 CLEAR 시작

`i_sw : 10 → 00 · 01 · 10 · 11`

CLEAR
→ STOP
→ RUN
→ STOP
→ CLEAR

k = 2 RUN 시작

`i_sw : 01 → 00 · 01 · 10 · 11`

RUN
→ STOP
→ RUN
→ STOP
→ CLEAR

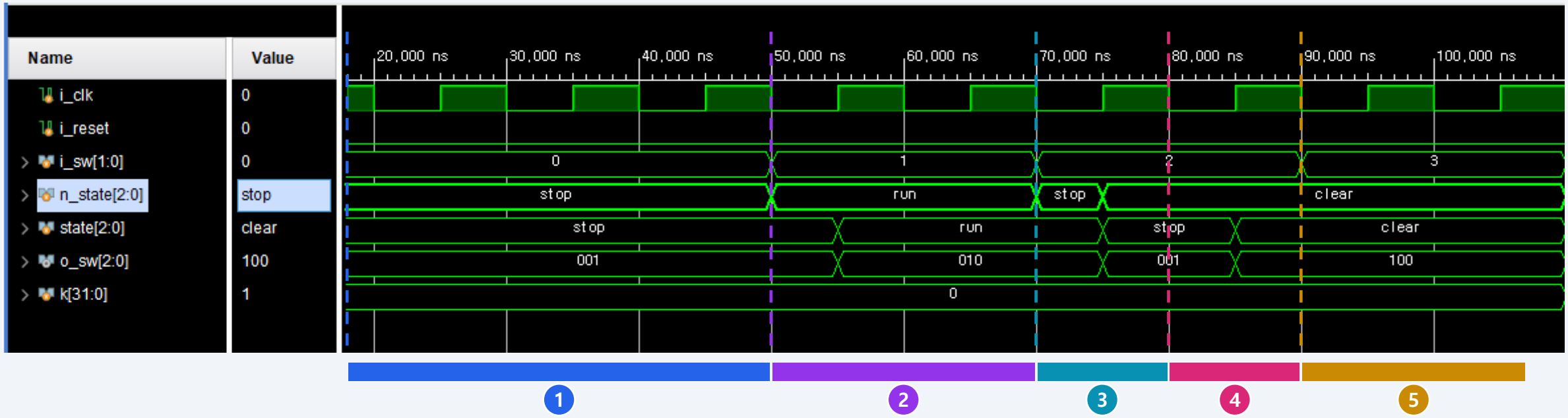
기존 TB가 지나간 arc (state × i_sw, 실측)

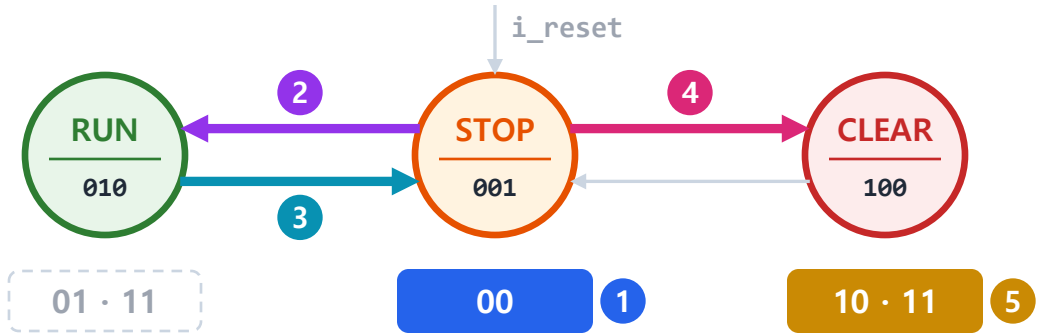
state	00	01	10	11
STOP	✓	✓	✓	✗ 미검증
RUN	✓	✓	✓	✗ 미검증
CLEAR	✓	✗ 미검증	✓	✓

arc 9 / 12

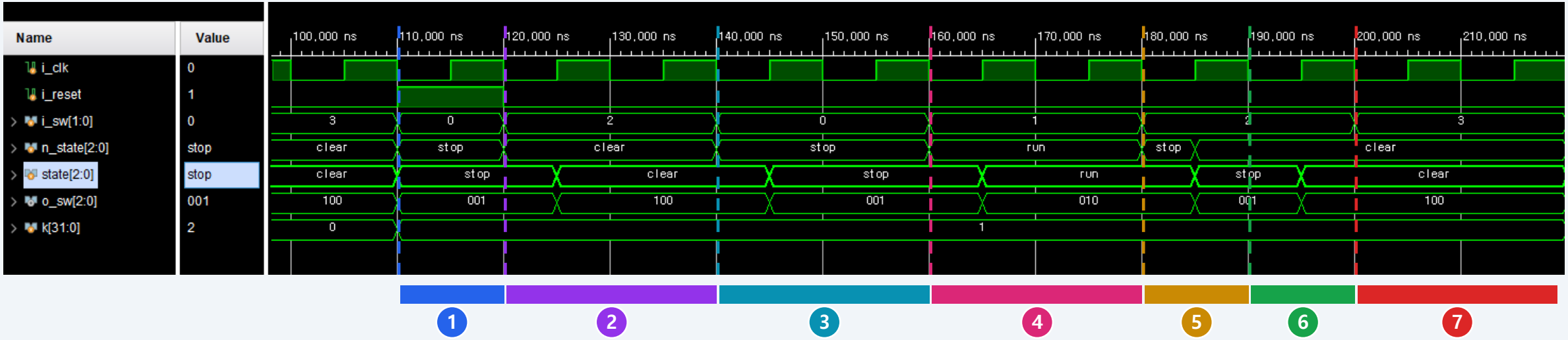
- STOP + 11 (SW0 우선 규칙) 미검증
- 기대값 · 자동 판정 없음

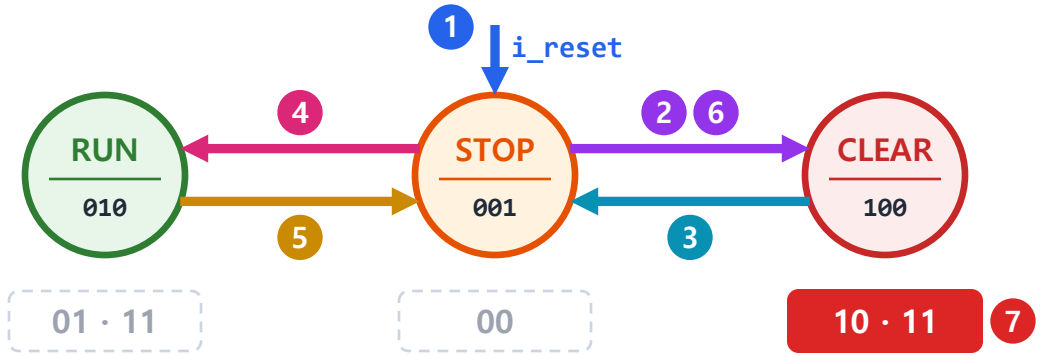
Scenario 1 Waveform Follows the Same Path



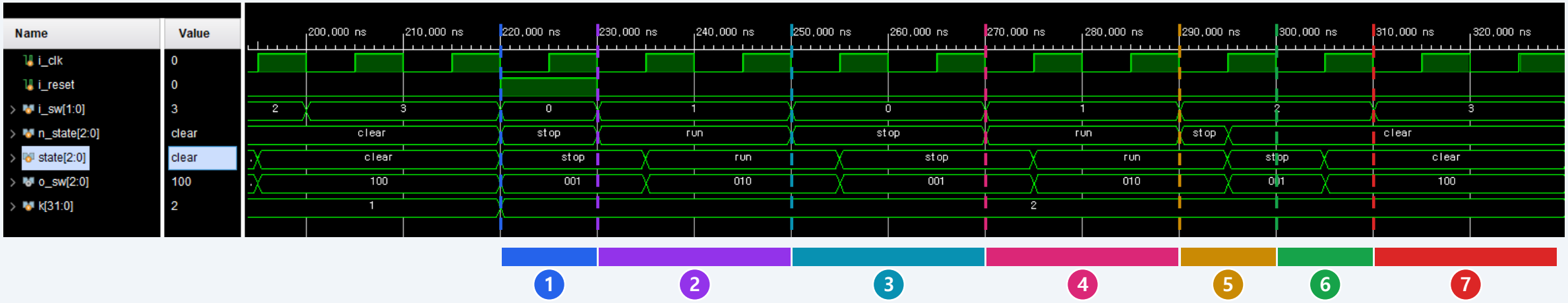


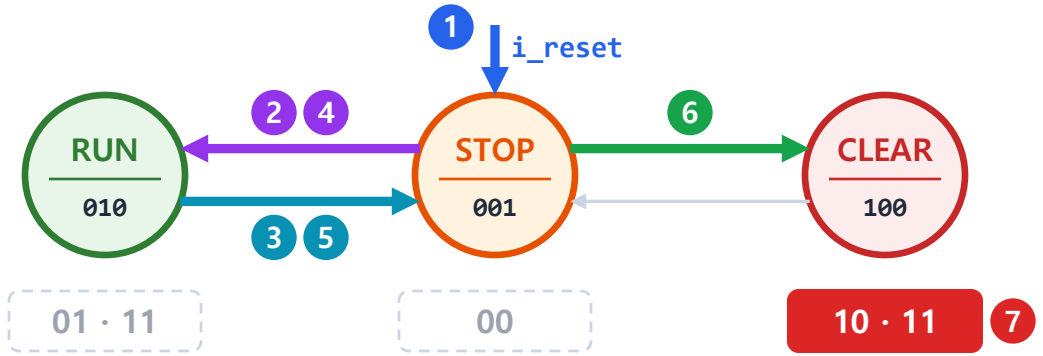
Scenario 2 Waveform Follows the Same Path





Scenario 3 Waveform Follows the Same Path





Existing TB Scores 21 / 40

평가 항목	점수	근거
입력 커버리지	**2 / 5**	arc 9 / 12, STOP + 11 · RUN + 11 · CLEAR + 01 미검증
기대값 독립성	**1 / 5**	기대값 없음, 정답 기준 = 작성자 머릿속 State Diagram
관측 지점	**4 / 5**	출력 포트 o_sw 파형 관측, 내부 state도 함께 표시
판정 타이밍	**4 / 5**	negedge input 변경 → 상승 엣지 시점 input 안정
자동 판정 · 리포트	**1 / 5**	err 카운트 · PASS/FAIL 없음, phase 번호 출력만
Reset 검증	**2 / 5**	phase 사이 reset 사용, 비동기 동작 (엣지 없는 시점) 미확인
보드 Top 일치	**4 / 5**	DUT = 보드에 올린 FSM 모듈 그대로
문서화 · 재현성	**3 / 5**	task · phase 구조와 주석 명확, SV 컴파일 시 k 선언 순서 오류

종합

21 / 40

등급 C

- stimulus 설계는 양호, 판정이 전부 사람 몫
- 결함 7종 중 3종은 원본과 파형 동일 → 육안으로도 검출 불가 (p.48)
- 보완 방향: Golden model + 자동 비교 + arc 커버리지 (p.49)

Three of Seven Faults Leave Identical Waveforms

09

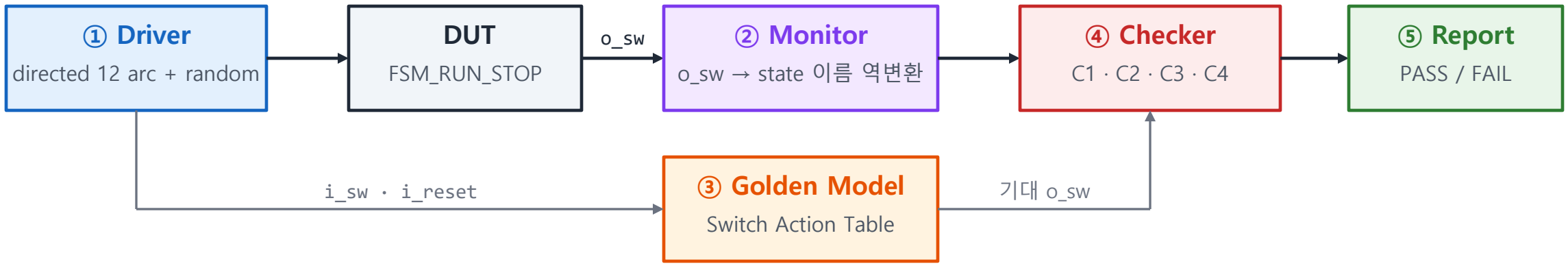
#	결함 (증상)	실제 버그 유형	기존 TB (파형 차이)	새 TB (error 수)
F1	STOP + 11 → CLEAR로 감	우선순위 뒤바뀜	✗ 파형 동일	✓ 78
F2	RUN 탈출 조건이 SW1 기준	비트 인덱스 오타	△ 7 cycle 차이	✓ 86
F3	CLEAR에서 LED1 점등 (010)	출력 코드 오류	△ 10 cycle 차이	✓ 79
F4	reset 후 CLEAR로 시작	reset 값 오류	△ 4 cycle 차이	✓ 37
F5	CLEAR + 01 → RUN 직행	천이 추가 (STOP 생략)	✗ 파형 동일	✓ 17
F6	o_sw가 n_state 기준 (Mealy화)	출력이 input 즉시 반영	△ 3 cycle 차이	✓ 266
F7	reset이 클럭 엣지까지 대기	동기 reset으로 작성	✗ 파형 동일	✓ 18
원본	FSM_RUN_STOP	-	-	PASS · 0

✗ 파형 동일
기존 TB stimulus로는 원본과 출력이 같음 → 육안 검출 불가

△ 파형 차이
차이는 존재, 기대값 없이 사람이 찾아야 검출

✓ 자동 검출
error 카운트 + FAIL 리포트

New TB Compares Output Pins Against Spec Table



① Driver

- reset → 목표 state 이동 → sw 인가, 3 state × 4 sw = 12 arc 전수
- 이후 random 300 cycle + 임의 시점 async reset

② Monitor

- 출력 포트 o_sw만 관측, 내부 state 미사용
- 001 · 010 · 100 → STOP · RUN · CLEAR 역변환

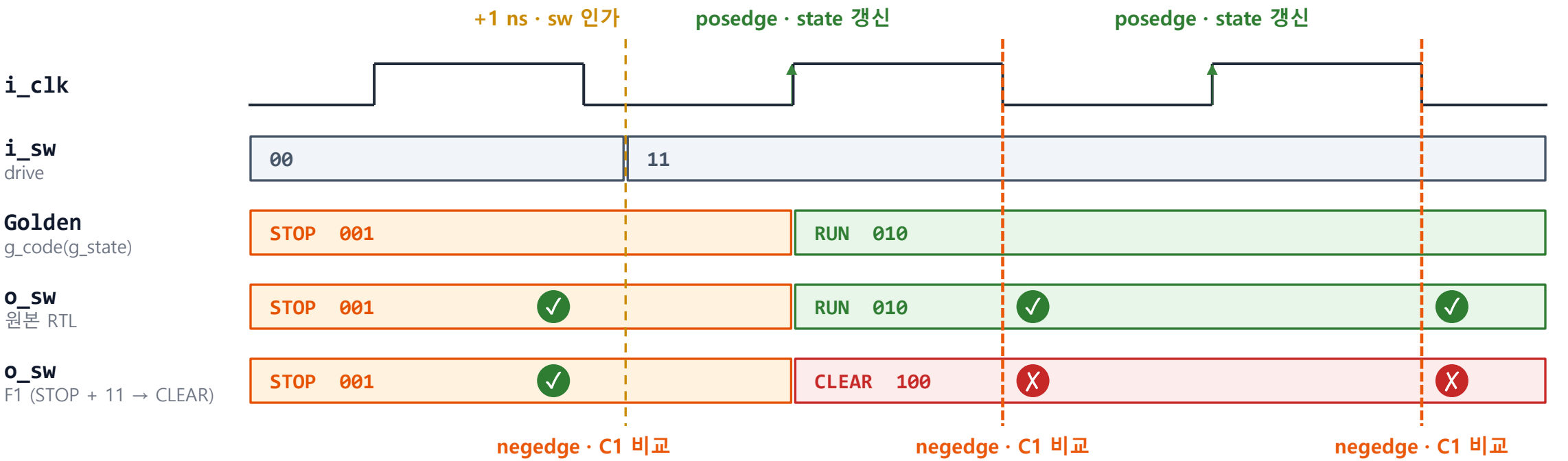
③ Golden Model

- DUT 코드 재사용 없이 표 기준으로 별도 작성
- DUT와 같은 엣지 · 같은 reset으로 기대 state 갱신

④⑤ Checker · Report

- C1 ~ C4 위반 시 error 카운트 + 시각 · 기대값 · 실제값 출력
- arc 커버리지 표 + PASS / FAIL 요약

1 o_sw == golden value ?



판정 시점

- 매 negedge (posedge 갱신 5 ns 뒤)
- i_reset = 0 구간만 판정

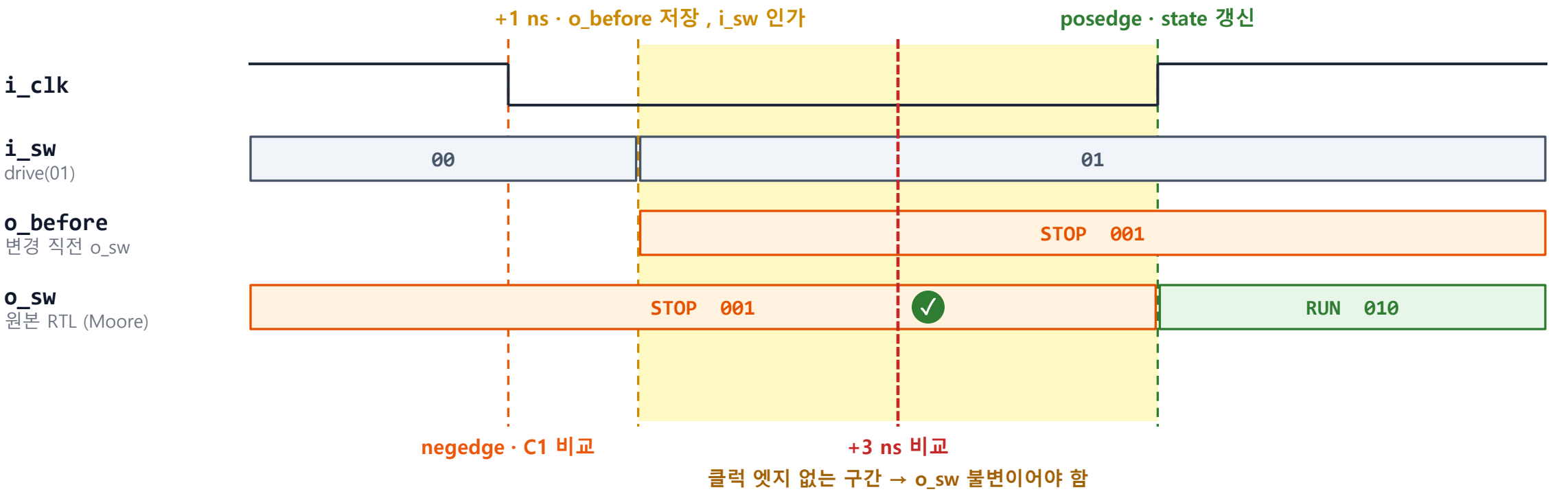
PASS 조건

- o_sw === g_code(g_state)
- DUT 외부의 기댓값과 비교하여 결과가 다르면 err_cnt1 ↑

잡는 결함

- F1 우선순위 · F2 비트 오타 · F3 출력 코드
- F4 reset 값 · F5 STOP 생략

2 o_sw == o_before (inner clock range)



판정 시점

- negedge + 3 ns (sw 인가 2 ns 뒤)
- 다음 posedge 이전

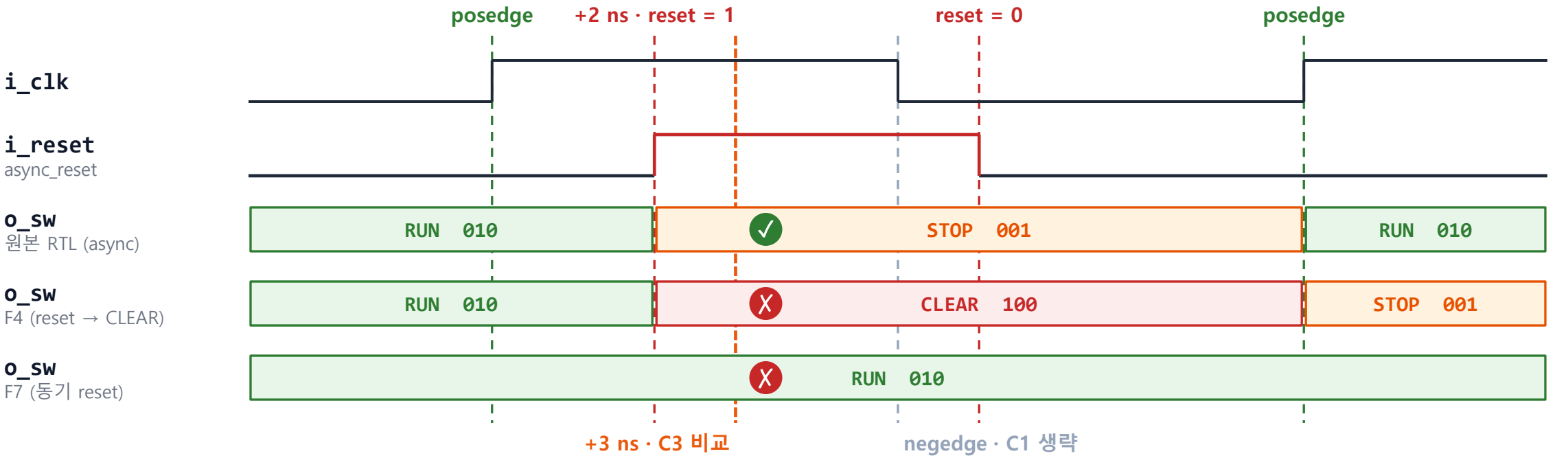
PASS 조건

- o_sw === o_before
- 입력만 바뀐 구간에서 출력이 불변하는지 검토

잡는 결함

- F6 Mealy화 (o_sw = n_state 기준)
- input → output 조합 경로

3 Pose edge 이후 Stop state로 넘어가는지 확인



판정 시점

- posedge + 3 ns (reset 인가 1 ns 뒤)
- reset 구간 = posedge + 2 ~ 6 ns, posedge 없음

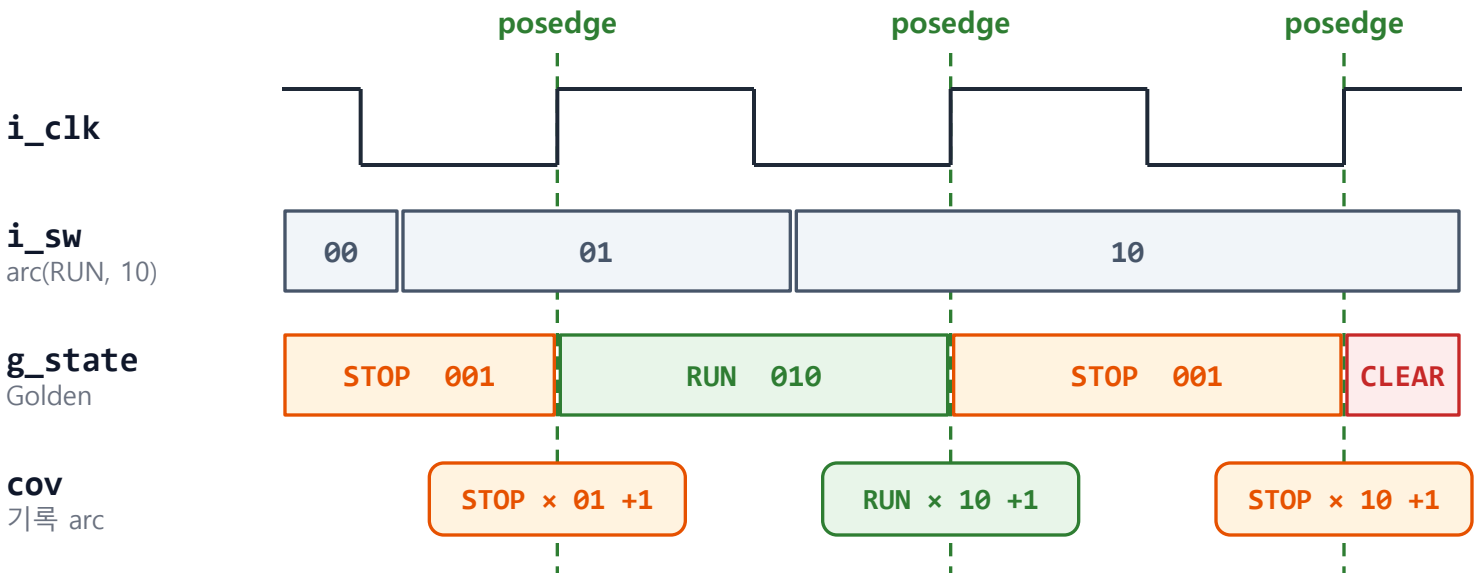
PASS 조건

- o_sw === 3'b001 (STOP)
- 클럭 엣지 없이 즉시 STOP

잡는 결함

- F4 reset 값 오류 (CLEAR 100)
- F7 동기 reset (posedge 없음 → reset 미반영)

4 Logs Each Arc at the Posedge and Requires 12 of 12



posedge 직전 값으로 기록 · `cov[g_state][i_sw] + 1`

Arc Coverage (state × i_sw)

state	00	01	10	11
STOP	0	+1	+1	0
RUN	0	0	+1	0
CLEAR	0	0	0	0

종료 시 0인 칸 존재 → FAIL
arc(RUN, 10) 1회 = 3칸 기록

기록 · 판정 시점

- 매 posedge 기록
- 시뮬레이션 종료 시 12칸 집계

PASS 조건

- 12칸 모두 1회 이상 실행
- hit == 12

검사 대상

- DUT가 아닌 TB stimulus
- 미실행 arc = 검증 누락

Golden Model Rewrites the Spec Table, Not the RTL


```
localparam G_STOP = 0, G_RUN = 1, G_CLEAR = 2;                          ①

function integer g_next(input integer st, input [1:0] sw);
  begin
    case (st)
      G_STOP : g_next = (sw[0]) ? G_RUN : (sw[1]) ? G_CLEAR : G_STOP;   ②
      G_RUN  : g_next = (sw[0]) ? G_RUN   : G_STOP;                     ③
      G_CLEAR: g_next = (sw[1]) ? G_CLEAR : G_STOP;                     ③
      default: g_next = G_STOP;
    endcase
  end
endfunction

function [2:0] g_code(input integer st);
  begin
    case (st)
      G_STOP : g_code = 3'b001;                                         ④
      G_RUN  : g_code = 3'b010;                                         ④
      G_CLEAR: g_code = 3'b100;                                         ④
      default: g_code = 3'bxxx;
    endcase
  end
endfunction
```

1 TB 전용 state 번호

- STOP · RUN · CLEAR = 0 · 1 · 2
- DUT enum (CLEAR = 0, STOP = 1, RUN = 2)과 별개
- DUT 인코딩 실수 비복사

2 STOP: SW0 먼저 검사

- 삼항 연산 순서 = 우선순위 SW0 > SW1
- sw = 11 → RUN 기대
- F1 (STOP + 11 → CLEAR) 검출 근거

3 RUN · CLEAR: 탈출 비트 하나만 확인

- RUN = SW0, CLEAR = SW1 기준
- F2 (비트 인덱스 오타) · F5 (STOP 생략) 검출 근거

4 g_code = 기대 o_sw

- state별 one-hot 코드 직접 기재
- F3 (CLEAR 출력 코드 오류) 검출 근거

Golden Steps with the DUT, C1 Compares Half a Cycle Later


```
always @(posedge i_clk or posedge i_reset) begin                ①
  if (i_reset) g_state <= G_STOP;                               ①
  else begin
    cov[g_state][i_sw] = cov[g_state][i_sw] + 1;                 ②
    g_state <= g_next(g_state, i_sw);                           ①
  end
end

always @(negedge i_clk) begin                                   ③
  if (!i_reset) begin
    cycles = cycles + 1;
    if (o_sw !== g_code(g_state)) begin                         ④
      err_c1 = err_c1 + 1;
      if (err_c1 <= 10)                                         ⑤
        $display("[C1 FAIL] ...");
    end
  end
end
```

1 DUT와 같은 clock · async reset

- 기대 state를 DUT와 같은 엣지에서 갱신
- reset 중 기대값 = STOP 고정

2 arc 실행 횟수 기록

- 현재 state × i_sw 칸 +1
- C4 arc 커버리지 표의 원천 data

3 negedge 비교

- posedge 갱신 후 반 cycle, o_sw 안정 구간
- i_sw 변경 (negedge + 1 ns) 이전 판정

4 !== (4-state 비교)

- X · Z까지 불일치 처리
- != 사용 시 X 비교 결과 = X → FAIL 누락

5 로그 10건 제한

- 출력만 제한, error 카운트는 계속 증가

Drive Task Applies the Switch and Runs the Moore Check


```
reg [2:0] o_before;

task drive(input [1:0] sw);
  begin
    @(negedge i_clk);                          ①
    #1;                                        ②
    o_before = o_sw;                           ③
    i_sw     = sw;                             ③
    #2;                                        ④
    if (o_sw !== o_before) begin               ④
      err_c2 = err_c2 + 1;
      if (err_c2 <= 10)
        $display("[C2 FAIL] ...");
    end
  end
endtask
```

1 negedge 대기

- 입력 변경 = 항상 negedge 이후
- posedge 시점 i_sw 반 cycle 이상 안정

2 #1 지연

- 같은 negedge의 C1 비교 먼저 실행
- C1 판정값과 i_sw 변경의 race 제거

3 변경 직전 출력 저장 → i_sw 인가

- o_before = 입력 변경 전 o_sw

4 #2 후 o_sw 재확인 (C2)

- 클럭 엣지 없는 구간 → Moore 출력 불변
- o_sw 변화 = input → output 조합 경로 (Mealy)
- F6 (o_sw가 n_state 기준) 검출 근거

drive 1회 = 1 clock · 입력 인가 + C2 판정 동시 수행

Every Arc Test Starts from a Checked Async Reset


```
task async_reset;
  begin
    @(posedge i_clk); #2;                          ①
    i_reset = 1; #1;                               ①
    if (o_sw !== 3'b001) begin                     ②
      err_c3 = err_c3 + 1;
      $display("[C3 FAIL] ...");
    end
    @(negedge i_clk); #1;                          ③
    i_reset = 0;                                   ③
  end
endtask

task arc(input integer st, input [1:0] sw);
  begin
    async_reset;                                   ④
    if (st == G_RUN)   drive(2'b01);               ④
    if (st == G_CLEAR) drive(2'b10);               ④
    drive(sw);                                     ⑤
    drive(sw);                                     ⑤
  end
endtask
```

1 posedge + 2 ns에 reset 인가

- 다음 엣지까지 클럭 엣지 없는 구간

2 1 ns 뒤 o_sw = 001 확인 (C3)

- async reset → 즉시 STOP
- F7 (동기 reset) → 이전 값 유지 → FAIL

3 negedge + 1 ns에 reset 해제

- 해제 시점을 posedge와 분리 → race 제거

4 목표 state 이동

- STOP: 이동 없음 · RUN: 01 · CLEAR: 10
- reset 직후 출발 → 시작 state 확정

5 같은 sw 2회 인가

- 1회차: 목표 state × sw arc 실행
- 2회차: RUN + 10 → STOP → CLEAR 경로 포함

Twelve Directed Arcs First, Then 300 Random Cycles


```
initial begin
  i_clk = 0; i_reset = 1; i_sw = 2'b00; g_state = G_STOP;
  err_c1 = 0; err_c2 = 0; err_c3 = 0; cycles = 0; seed = 1008;   ①
  ...
  #12 i_reset = 0;

  $display("[MARK] %0d ns phase 1 : directed arcs", $time);      ①
  for (s = 0; s < 3; s = s + 1)                                  ②
    for (w = 0; w < 4; w = w + 1)                                ②
      arc(s, w[1:0]);                                            ②

  $display("[MARK] %0d ns phase 2 : random 300 cycles", $time);
  for (i = 0; i < 300; i = i + 1) begin                          ③
    if (($random(seed) & 31) == 0) async_reset;                  ③
    drive($random(seed));                                        ③
  end
  ...
  if (err_c1 + err_c2 + err_c3 == 0 && hit == 12)                ④
    $display(" RESULT : PASS");
  else
    $display(" RESULT : FAIL (errors=%0d)", ...);
  $finish;
end
```

① seed = 1008 고정 · [MARK] 로그

- 매 실행 같은 random 순서 → FAIL 재현
- [MARK] = 파형 캡처 위치 표시

② Phase 1 · directed 12 arc

- 3 state × 4 sw 전수, arc task 호출
- random 결과와 무관하게 C4 12 / 12 확보

③ Phase 2 · random 300 cycle

- 사람이 고르지 않은 입력 순서
- & 31 == 0 → 1/32 확률 async reset
- 임의 state에서 C3 판정

④ PASS 조건

- C1 · C2 · C3 error 0 + C4 arc 12 / 12
- 미달 시 FAIL + error 수 출력

Five Steps Turn an FSM Spec into a Self-Checking TB

10

1 Spec 표 → Golden

v2 구현

```
g_next · g_code function
```

다른 FSM 적용

- state × 입력 표를 function으로 작성
- DUT 코드와 다른 형태 (삼항 · 표)
- 출력 코드 직접 기재

관련 Check

C1 기대값

2 Golden 갱신

v2 구현

```
always @(posedge i_clk or posedge i_reset)
```

다른 FSM 적용

- DUT와 같은 clock · reset 조건
- 동기 reset DUT일 경우 동기 형태
- arc 실행 횟수 기록

관련 Check

C1 기대 state · C4 표

3 Checker 시점

v2 구현

```
negedge · !== 비교
```

다른 FSM 적용

- 입력 · 출력 안정 구간에서 판정
- 출력 포트만 관측
- X · Z까지 비교

관련 Check

C1

4 Driver task

v2 구현

```
drive · async_reset · arc
```

다른 FSM 적용

- 입력 인가 + 구조 검사 묶음
- Moore 불변 · async reset 확인
- 1 call = 1 clock

관련 Check

C2 · C3

5 Stimulus · Report

v2 구현

```
12 arc → random 300 → 표
```

다른 FSM 적용

- directed 전수 arc 먼저
- random + 고정 seed
- coverage 표 · error 수 · PASS/FAIL

관련 Check

C4 · PASS / FAIL

New TB Passes with 12 / 12 Arc Coverage

Arc Coverage (state × i_sw, 실행 횟수)

state	00	01	10	11
STOP	43	41	41	33
RUN	28	26	40	24
CLEAR	22	13	17	27

Check 결과 · 원본 RTL

항목	결과
checked cycles	333
C1 output mismatch	0
C2 Mealy path	0
C3 async reset	0
C4 arc coverage	12 / 12

PASS

원본 RTL · error 0

```
------------------------------------------------
Arc coverage (state x sw)   00   01   10   11
  STOP                      43   41   41   33
  RUN                       28   26   40   24
  CLEAR                     22   13   17   27
------------------------------------------------
checked cycles : 333
[C1] output mismatch : 0
[C2] Mealy path      : 0
[C3] async reset     : 0
[C4] arc coverage    : 12 / 12
RESULT : PASS
------------------------------------------------
```

Moore FSMs Verified by Self-Checking TB

결론

1 5-state FSM 보드 동작 확인

Homework-1 · FSM_S3 · Switch 입력에 따른 State 천이 → LED 출력

2 RUN · STOP · CLEAR 모듈 검증 및 합성

Homework-2 · FSM_RUN_STOP · Switch 입력에 따른 3-state Moore FSM

고찰

1 Golden model로 기대값 독립성 확보

- AI 검증 평가에서 기존 TB의 기대값 부재 확인
- DUT와 분리된 Golden model 설계 방법 터득

2 task 기반 Driver로 입력 커버리지 확보

- 입력 커버리지 확보를 위한 Driver 설계 방법
- task 사용법 습득

3 향후 과제 · async reset 판정

- clock edge와 동시에 인가되는 reset
- async reset deassert 구간에 들어오는 clock edge
- 두 경우의 simulation 판정 방식 연구